C++17 Iterating Problems

The C++ Book Only The Author Will Read

Daniel Rodriguez

C++17 - Iterating Problems

The C++ Book Only The Author Will Read

Daniel Rodriguez

Table of Contents

Dedication . . . 1
Introduction . . . 2
The Chapters and Their Content . . . 3
Navigating The Code . . . 4
Final Words . . . 4
1. Hello World . . . 5
1.1. Using an Output Iterator . . . 5
1.2. Adding Input Iterators . . . 6
1.3. Adding an STL Algorithm . . . 7
1.4. A Custom Output Iterator . . . 8
1.5. An STL-like Solution . . . 11
1.6. Summary . . . 11
2. Input and Output . . . 12
2.1. Checking the Nature of the Iterators . . . 14
2.2. Summary . . . 20
3. Input And Output - Reloaded . . . 21
3.1. A Better Input And Output . . . 21
3.1.1. Deducting Types . . . 21
3.1.2. Regaining The “auto” Return Type . . . 23
3.1.3. SFINAE On Steroids . . . 25
3.1.4. Leaner and Meaner . . . 30
3.2. Summary . . . 32
4. Hello World - Revolutions . . . 33
4.1. SFINAE for the custom iterator . . . 33
4.2. Checking Type Compatibility . . . 35
4.3. Summary . . . 39
5. Types, Types, Types . . . 40
5.1. The Obvious Approach . . . 40
5.2. The Classic Recursive Template . . . 41
5.3. The Classic Template Reworked . . . 44
5.4. The “if constexpr” Express . . . 45

5.5. The “if constexpr” Single Method Solution . . . 47
5.6. The Fold Expression Miracle . . . 48
5.7. The Fold Expression Single Method . . . 50
5.8. The Last Crown Jewel - std::variant . . . 53
5.9. Summary . . . 54
6. Types - Resurrected . . . 56
6.1. Iterating Iterators . . . 56
6.2. Templated Iterators . . . 56
6.3. Criticism . . . 63
6.4. Dynamic Type Output . . . 63
6.5. Going The Iterating Mile . . . 69
6.6. Summary . . . 74
6.7. Bonus Points . . . 74
7. For Loop . . . 81
7.1. Going Python . . . 81
7.2. Refining And SFINAE . . . 88
Colophon . . . 95

Dedication

Some nice dedication

Introduction

In our modern times and with the advent of *Artificial Intelligence*, there are many prophets that have loudly announced the end of programmers as a species. And they are probably right. The exact moment in time still remains a mystery.

Until then, a hobby programmer, like your humble author, can still decide to tackle some programming challenges using C++. This is something I have done in the past to keep myself updated with the incredible changes that have introduced into the language since the C++11 standard was brought to life.

Considering that the last serious period of time was back in 2016, it was already time to bring myself up to date. My first intention was to use *CodeEval*, the platform I used in the past, but the platform is gone for good. The second obvious choice was *HackerRank*, simply because I tested it a couple of times and my account was still there.

Seeing C++20 as one of the choices, skipping the C++17 wave seemed like a good choice. Unfortunately, the C++20 selection was simply a label from a `clang` compiler version with a couple of features from the end of 2017, and for sure not the next best standard sanctioned in 2020.

On the bright side, *HackerRank* does not lock the editor for this C++20 label. It does in some, not all, instances of problems for previous versions. This is something I could use to my advantage to provide complete solutions, rather than solving the problem by filling out just one method.

This gave me an idea, that later lead to this book/blog/static site, call-it-x if you wish. I could rethink the problems and address them as something much bigger, forcing myself along the way to work with as many features of C++17 as possible. And as it usually happens in life, one idea led to the next: solve the challenges using iterators as much as possible.

The STL (Standard Template Library) was already a thing when I had my first contact with C++, even if C++98 was yet to be conceived. Back then, I hit a wall when I first tried to love iterators, until one day something made *"click"* and the idea seemed simply but incredibly brilliant. Hence the thought of solving everything now with iterators, even a simple *"Hello, World!"* challenge.

The Chapters and Their Content

Each chapter presents several solutions to a given challenge. Even more, a challenge can be revisited in a second chapter to go above and beyond. Usually, the simplistic, obvious and boring solution is presented first as the starting point. After that, several solutions are presented, showing different approaches and/or the evolution to a generic iterator based solution.

Not all the solutions are meant to succeed in solving the problem. Some of them are meant to show where a human being, the author, fails at quickly identifying what one must do.

If a solution introduces new concepts, be it from C++17 or previous standards, it is specifically mentioned. That is the whole point of the exercise, apply a variety of techniques and the tools of the C++ arsenal, embedding that knowledge deep in one's brain.

I have obviously read blogs and quickly scanned books and seen online C++ online reference guides over the years. And I have used the reference guides during the writing of this text. Because I doubted how to do a correct usage of something, to see alternatives and to explore the member functions.

But I want to say something: if I have not understood a concept or technique, I have not used it. Copying code is something even AI can do, but that would not bring me personally any further. It is not about solving the problem with someone else's code and idea, it is about doing it myself, because I understand how to do things.

Savvy C++ programmers will obviously identify several areas of improvements, obvious error and even things that work due to sheer luck. They should not hesitate to provide feedback on the repository as that will bring one step closer to better understand and use C++. As a hobby programmer of course, but a programmer nonetheless.

> in some cases I simply left things a bit untidy, because my other life (the real one) has a higher priority and an incredible gravitational pull. Please, be lenient with this humble coder.

Let me finish by saying that writing this in the form of a book has really helped me. Telling a story and wanting to improve the solutions to be generic and iterator based, has improved my focus and made me a better C++ hobby programmer++. Or so do I believe, because as a kid I thought I was so good I could play professional soccer and that never ever crystallized.

Navigating The Code

The code for the chapters follow the chapter numbering and has this form: `xy-descriptive-text`, where `xy` is a series of two digits that matches the chapter in which the solutions in that folder are developed and shown.

Going inside the folder each solution has also this form: `descriptive-text-ab.cpp`. Once again the `##` placeholder stands for two digits, following the order in which the solutions are presented in the chapter.

One can test each solution inside the folder with: `make ab`. This will compile `descriptive-text-ab.cpp` and execute the binary `descriptive-text-ab` (with `.exe` under *Windows*), feeding it a test case and evaluating the output against the well-known result.

Beware: the `Makefile` uses *GNU Make* extensions, to look for files, parse names, generate the tare

A failed test case will show the expected output, the actual output and the difference of the two. This can be forced for positive test cases with `make show ab`. If in doubt, just run `make` or `make help` to see the available options (like an obvious `make all` to compile and test all solutions)

That was for each individual challenge. Executing `make xy` from the top directory, will enter the `xy-descriptive-text` directory and compile and test all solutions `make all`. At this level a `make all` is also possible to compile and test all the solutions for all the challenges.

If you add a new solution following the aforementioned conventions, the `Makefile` will automatically be able to compile and test it. That is: `make ij` will work, where `ij` are the digits you will have assigned to your solution with this convention: `xy-descriptive-test/descriptive-test-ij.cpp`.

Final Words

Hopefully someone finds something interesting here and not only the author enjoys the content and the process of having crafted it.

1. Hello World

Title: Hello, World!
Link: https://www.hackerrank.com/challenges/cpp-hello-world

Getting started with any programming language always implies a "Hello World" example and Hackerrank also has this challenge to get problem solvers started with C++. Solving the problem is obvious, but the goal was not to make obvious things. Rethinking the problem to make something useful out of it is the real challenge. Let us see how to rework this into something interesting.

We first draft the obvious solution, considering that the expected output is: *"Hello, World!"*.

```
#include <iostream> // std::cout

int
main(int, char *[]) {
    std::cout << "Hello, World!";
    return 0;
}
```

01-hello-world/hello-world-01.cpp

There it is. The solution does not even require a `\n` (or `std::endl`) after printing the message. But notice that a comma, followed by a space, is used as a delimiter unlike in many other cases. That will be our first challenge.

1.1. Using an Output Iterator

Because the goal is to solve the problems, where possible, with a Standard Library approach, the solution above is not really satisfying. Let us therefore introduce a classic from the STL: iterators. We will be adding an output iterator to the mix, in this case: `std::ostream_iterator`, that will take over the functionality of `std::cout`.

```
#include <iostream> // std::cout
#include <iterator> // std::ostream_iterator

int
main(int, char *[]) {
    auto out = std::ostream_iterator<std::string>{std::cout};
```

```
    *out++ = "Hello, World!";
    return 0;
}
```

01-hello-world/hello-world-02.cpp

We have added our output iterator, initially managing it manually with the `*` (dereference) and `++` (postfix increment) operators to output our string for us. This still bears no resemblance to an STL solution. Luckily, our `std::ostream_iterator` can take a second parameter during construction, which is the delimiter to be used after each output operation. We can use it to get closer to the final goal.

Notice that we have used `std::string` with `std::ostream_iterator`. We could have used a good old `const char *` but our focus is the Standard Library.

1.2. Adding Input Iterators

Let us now simulate that there is also input, by managing *"Hello"* and *"World!"* as separate tokens and making the `, ` delimiter a part of the output operation and not part of the content itself.

```
#include <array> // std::array
#include <iostream> // std::cout
#include <iterator> // std::ostream_iterator

int
main(int, char *[]) {
    auto hello_world = std::array{"Hello", "World!"}; // input range
    auto out = std::ostream_iterator<std::string>{std::cout, ", "}; // with delim
    for(auto in = hello_world.begin(), last = hello_world.end(); in != last;)
        *out++ = *in++; // input to output, move both to next position
    return 0;
}
```

01-hello-world/hello-world-03.cpp

This is much better. By using an `std::array` to hold the two words in our message, we can use the `begin` and `end` methods to iterate over the input range and then output each element with our iterator. Even for this simple case, we obviously avoid checking against `end()` each and every time. Imagine if we did that for an array of millions of elements, instead of storing the `last` iterator in a variable.

We are using the C++17 `std::array` facilities, with which it is possible to forego the template syntax, avoiding the need to specify the type of the array elements and the array size. The compiler will do it for us. And this is yet another reason `std::string` was a good choice as the type for `std::ostream_iterator`. We could quickly find out if the compiler is choosing `const char *`, however by choosing `std::string` there is no need to worry, because it will be converted to a `std::string` when moving it to the output.

In any case this solution fails to pass the test because the output has a trailing comma.

```
Hello, World!,
```

An improvement is still possible, by using a range based `for` loop, instead of manually controlling the iteration.

```
#include <array> // std::array
#include <iostream> // std::cout
#include <iterator> // std::ostream_iterator

int
main(int, char *[]) {
    auto hello_world = std::array{"Hello", "World!"}; // input range
    auto out = std::ostream_iterator<std::string>{std::cout, ", "}; // with delim
    for(const auto &hw: hello_world) // range based for loop
        *out++ = hw; // val to output and move output
    return 0;
}
```

01-hello-world/hello-world-04.cpp

Some syntactic sugar that simplifies the look of our `for` loop and removes the direct use of the input iterators from the equation. The change implies no change for the result: still a failure due to the aforementioned trailing comma.

1.3. Adding an STL Algorithm

One of the goals was to use the STL as much as possible. And there is an ideal target for us: `std::copy`, which takes an input range and copies each element to a destination range, by taking an iterator that points to the first element of the output range. This destination iterator has to be at least an output iterator. We already have those elements at hand: an

input range and an output iterator.

```
#include <algorithm> // std::copy
#include <array> // std::array
#include <iostream> // std::cout
#include <iterator> // std::ostream_iterator

int
main(int, char *[]) {
    auto hello_world = std::array{"Hello", "World!"}; // input range
    auto out = std::ostream_iterator<std::string>{std::cout, ", "}; // with delim
    std::copy(hello_world.begin(), hello_world.end(), out); // copy range => out
    return 0;
}
```

01-hello-world/hello-world-05.cpp

Using `std::copy` has allowed us to simplify the solution. We no longer need to hold the result of `end()` in a variable and there is no need to manually operate on the iterators. The algorithm does all the `*` and `++` heavy lifting for us. This may seem like an overkill for this case, but the goal is to make use of the existing facilities. Reinventing the wheel can be a viable solution if an extreme optimization is needed. In any case, the people writing the algorithms for the STL, including several specializations, can write much better code than we do.

Unfortunately, the trailing comma is still there.

```
Hello, World!,
```

Expecting anything else would have been naive because `std::copy` is using our instance of `std::ostream_iterator`, as we did before.

1.4. A Custom Output Iterator

This is where extra thinking and development are needed to square the circle: a custom output iterator! Although one has to be humble and admit it will only be a wrapper around the `std::ostream_iterator` functionality. But what a mighty wrapper because it will take over the management of the delimiter.

```
template <typename T>
class os_iterator {
    std::ostream_iterator<T> m_iter; // wrapped iterator
    T m_delim; // delimiter between elements
    bool m_dodelim = false; // when to start separating

public:
    // needed for an iterator
    using iterator_category = std::output_iterator_tag;

    // constructor replicating the wrapped iterator's constructor
    os_iterator(std::ostream &os, const T &delim) : m_iter{os}, m_delim{delim} {}

    // no-ops because only the assignment (= operator does something)
    auto operator ->() { return this; }
    auto &operator *() { return *this; }
    auto &operator ++() { return *this; } // ++prefix
    auto &operator ++(int) { return *this; } // postfix++

    // Operation with the wrapped iterator, choosing when to output the sep
    auto &operator =(const T &outval) {
        if (m_dodelim) // output delimiter before the second/later elements
            *m_iter++ = m_delim;
        else
            m_dodelim = true;

        *m_iter++ = outval;
        return *this;
    }
};
```

01-hello-world/hello-world-06.cpp

In order to be useful, the wrapper has to be template based to mimic the original. It can then provide the operations that one will manually use, as we saw with `*out++ = val;`, and account for the possibility of other operations being used by the algorithm inside the STL operation. That is why both `++` increment operators, prefix, and postfix, are available. Those and the `*` dereference operator are simple NOP commands.

The real work is done with the `=` assignment operator, when the iterator is given the value that has to be output. That is where the decision to output the delimiter is done. And it is done just before outputting the value, for all assignments except for the first, which is

controlled with a boolean, `m_dodelim`. The STL outputs the delimiter after the fact, we do it before, changing the logic.

Anyone who also knows Python will notice that we are replicating the `'delimiter'.join(iterable of str)` functionality, that creates a string by joining the iterable with the delimiter in between elements, except after the last element. Even better, we do not need our range to be made out of strings, because of the generic functionality templates offer us. Granted, in Python the iterable of strings can be quickly achieved with `(str(x) for x in iterable)`.

Defining the `iterator_category` in our custom iterator is compulsory and the alias we set it to, is the obvious choice: `std::output_iterator_tag`, because that is what our custom iterator and the wrapped one are doing: output.

One final note about this custom iterator. The usual implementation of the `++` postfix operator looks like this.

```
    auto operator ++(int) { auto tmp = *this; ++(*this); return tmp; }
```

But because our `operator *()` and `operator ++()` are *NOP* implementation returning a reference to itself, creating a `tmp` copy is not needed and we can implement `++(int)` also as an *NOP*.

The new main function that goes with it looks like this.

```
int
main(int, char *[]) {
    auto hello = std::array{"Hello", "World!"};
    auto out = os_iterator<std::string>{std::cout, ", "};
    std::copy(hello.begin(), hello.end(), out);
    return 0;
}
```

01-hello-world/hello-world-06.cpp

This delivers the expected solution output, with no trailing `, ` delimiter at the end.

```
Hello, World!
```

But more importantly, our solution is just one step away from one of the declared goals: be STL-like. The general form we will be looking for is the one taking an input range and accepting also the beginning of a destination (or output) range.

1.5. An STL-like Solution

It may seem like something redundant to go any for it, because `std::copy` is already part of the STL, but let us go for our own STL-like solution.

```
template <typename I, typename O>
auto
hello_world(I first, I last, O out) {
    return std::copy(first, last, out);
}

int
main(int, char *[]) {
    auto hello = std::array{"Hello", "World!"};
    auto out = os_iterator<std::string>{std::cout, ", "};
    hello_world(hello.begin(), hello.end(), out);
    return 0;
}
```

01-hello-world/hello-world-07.cpp

Our newly defined `hello_world` has the final form we will be trying to use for each and every solution. We have obviously used `I` for *"Input"* and `O` for *"Output"* to make things as clear as possible.

1.6. Summary

A simple 1-statement solution has been turned into a basic model for future solutions. By converting the non-input (constant value) into an iterable input range, wrapping the output iterator into our own custom iterator, we are able to control the presentation and model the final solution function after the STL algorithms.

Mission accomplished! Well, one of many missions to come.

2. Input and Output

Title: Input and Output
Link: https://www.hackerrank.com/challenges/cpp-input-and-output

After the classic, output-only *"Hello, World!"*, we are confronted with first accepting input, performing then a simple manipulation of the data and sending finally the result to standard output. I.e.: read three integers from standard input, add them up and output the result.

The straightforward simplistic solution.

```
#include <iostream> // std:cin/cout

int
main(int, char *[]) {
    int i1, i2, i3;
    std::cin >> i1 >> i2 >> i3;
    std::cout << (i1 + i2 + i3);
    return 0;
}
```

02-input-and-output/input-and-output-01.cpp

Not really the challenge we were looking for. Let us immediately go for a first `auto` /`iterator` based version.

```
#include <iostream> // std:cin/cout
#include <iterator> // std::istream_iterator/ostream_iterator

int
main(int, char *[]) {
    auto in = std::istream_iterator<int>{std::cin};
    auto out = std::ostream_iterator<int>{std::cout};
    *out++ = *in++ + *in++ + *in++;
    return 0;
}
```

02-input-and-output/input-and-output-02.cpp

Unlike in the *"Hello, World!"* case, we have real input and that is why we introduce an

iterator to gather the input: `std::istream_iterator`. Just like its counterpart the `std::ostream_iterator`, it takes a stream (an input one, obviously) and reads characters that will be converted to the type used in the template. Whitespace, including newlines, and End-of-file are used as the separators to discriminate the input values.

The solution is not really satisfying, even if we can use iterators and apply `auto` to all variable declarations. It is still hardcoded to take three `int` values. Notice that we do not need to even store the values, thanks to the use of the input iterator.

Moving forward is straightforward: if there is an input iterator, there must, for sure, be a counterpart marking the end of the stream. Recall that our input range in the previous solution had `begin`/`end` member functions precisely for that. Using here an *end-of-stream* iterator we could read integers until the input is exhausted, making the solution dynamic.

Such *end-of-stream* input iterator is instantiated by not passing a stream to the constructor, i.e.: `std::istream_iterator<int>();`.

At the same time, we can use the proper algorithm from the STL to add up all integers: `std::accumulate`. To spare us some boredom and as we did with the *"Hello, World!"* problem, we will move the core of our solution, even if only a wrapper, directly to a generic function.

```
#include <iostream> // std:cin/cout
#include <iterator> // std::istream_iterator/ostream_iterator
#include <numeric> // std::accumulate

template<typename I, typename O>
auto
input_and_output(I first, I last, O out, int init = 0) {
    *out++ = std::accumulate(first, last, init);
}

int
main(int, char *[]) {
    auto in = std::istream_iterator<int>{std::cin};
    auto in_last = std::istream_iterator<int>{};
    auto out = std::ostream_iterator<int>{std::cout};
    input_and_output(in, in_last, out);
    return 0;
}
```

02-input-and-output/input-and-output-03.cpp

For the sake of it, we may even go for a single statement solution inside of `main`.

```
#include <iostream> // std:cin/cout
#include <iterator> // std::istream_iterator/ostream_iterator
#include <numeric> // std::accumulate

template<typename I, typename O>
auto
input_and_output(I first, I last, O out, int init = 0) {
    *out++ = std::accumulate(first, last, init);
}

int
main(int, char *[]) {
    input_and_output(
        std::istream_iterator<int>{std::cin},
        std::istream_iterator<int>{},
        std::ostream_iterator<int>{std::cout}
    );
    return 0;
}
```

02-input-and-output/input-and-output-04.cpp

This is a lot more like it. If you look at how this is usually solved, the problem solver will first copy the input, to an `std::vector`, which will then be used as the input range for `std::accumulate`. I.e.: an input range is used to create another input range, that is then used for the solution. It really seems we are better off by directly using the available input range.

2.1. Checking the Nature of the Iterators

Stopping here would go against the goal of our quest, i.e.: learning, learning more, learning even mor, be better and write better code. Let us therefore carry on by adding some checking for our template parameters with *SFINAE*.

When I first heard about *SFINAE* I had to make sure I was getting it right: *"Substitution Failure Is Not An Error"*. Because once I managed to wrap my head around it, I thought that *SFIAFWAD* would have been a better acronym: *"Substitution Failure Is A Feature*

Working As Designed". No matter in which industry you have worked, it is quite often that an error is deemed to be a feature and tagged with the label, "it works as designed". It is anyhow highly unlikely that my proposal would have probably made the cut for the naming.

Let us see why we would have to use *SFINAE*. In our two solutions, we have modeled our functions in this manner.

```
template<typename I, template typename O>
auto
solution_function(I first, I last, O out) {
    std::do_something(first, last, out);
    // or *out++ = std::do_something(first, last);
}
```

Typical Solution Model

Our STL-like `solution_function` accepts the (until compilation) unknown types `I` and `O` and pass them completely unchecked to `std::do_something`, or feeds the result of the latter to the output iterator.

Inside the function, it is assumed that `I` is an input iterator and `O` is an output iterator, but we have no certainty if those assumptions are actually true. Compilation will most probably fail at some point if they are not, but it may also go through causing a spectacular runtime.

We could do the following and our on-the-fly checking IDE will not complain: `solution_function(5, 6, out);`. Those two integers are, obviously, no iterators and it would be nice to catch the error as soon as possible, ideally whilst typing.

Here is where *SFINAE* comes to the rescue. By imposing some restrictions and having the parameters checked during compilation or by our IDE, the iterators will be checked to see if they are really what they claim to be. *SFINAE* works by removing potential template candidates. There will be no match for our call if the arguments do no fit our restrictions.

Yes, C++20 introduces *"concepts"* but we are still using C++17, so we have to settle for *SFINAE*, the next best thing after sliced bread. Our template-based `solution_function` will be enabled if some conditions are met. To implement that *enabling* (or *disabling*) we will be using `std::enable_if` as the basis. This utility function will fail if the boolean conditions passed to it are not met. But if the conditions are met, we will be able to use the type

passed to it as the result. The general form is: `std::enable_if<condition, enabler>`, where in this case the *enabler* will be used as the return type of our `solution_function` algorithm.

To use the `enabler` as a type, we have two options:

- `std::enable_if<condition, enabler>::type` (C++11)
- `std::enable_if_t<condition, enabler>` (C++14)

We will go for the latter because it seems cleaner, `_t` for `::type`, and because C++17 introduced more of these helpers as we will see below, i.e.: others did also see the benefit in using the `_x` suffix as opposed to `::whatever_we_want`.

The most obvious idea is checking if the parameters bear the proper iterator tags for their intended functionality. Let us add some vertical spacing here or the sake of clarity. We are going directly for the helper with `_v` for `::value` when using `std::is_same`.

```
template <typename I, typename O>
std::enable_if_t<
    // enable if condition
    std::is_same_v<
        std::input_iterator_tag,
        typename I::iterator_category>
    &&
    std::is_same_v<
        std::output_iterator_tag,
        typename O::iterator_category>
    ,
    void
    > // get the type of the enable_if result
solution_function(I first, I last, O out) {
    std::do_something(first, last, out);
}
```

SFINAE - Checking if something is exactly something

`std::is_same_v`, a C++17 present, will be substituted with `true` or `false` during compilation. If the logical `and` test, of those conditions, has a single `false`, `std::enable_if_t` will fail and the candidate will be removed from the set of possible matches for an invocation of `solution_function`.

This unfortunately has some problems.

- We are checking with `std::is_same_v<T>` (aka `std::is_same<T>::value`) and our iterator could be not only an input iterator, but actually a lot more. A *RandomAccessIterator* is also an *InputIterator* that can do a lot more things.
- We check directly `I::iterator_category` and `I` (or `O`) could be a simple type like a `const char **`, that the STL can manage as an iterator. Compilation failure knocking on the door in 3, 2, 1.

Luckily for us, both issues have a solution.

- Run the check with `std::is_base_of`. The function will check inside `std::input_iterator_tag` to see if it is the base class of whatever tag the iterator has declared in its implementation. This will then be a positive check not only for an *InputIterator* but for others too, such as the aforementioned *RandomAccessIterator*.
- Use `std::iterator_traits<T>` to look for the tag. This STL functionality understands what is and what can be an iterator and offers a unified interface to retrieve the actual declared `iterator_category` tag or the implied one in case of thingies such as `const char **`.

Let us apply all this to our `input_and_output` challenge. Using the `_v` variant of `std::is_base_of`, of course.

```
#include <iostream> // std:cin/cout
#include <iterator> // std::istream_iterator/ostream_iterator
#include <numeric> // std::accumulate
#include <type_traits> // std::iterator_traits/enable_if/is_base_of

template <typename I, typename O>
std::enable_if_t<
    // enable if condition
    std::is_base_of_v<std::input_iterator_tag,
        typename std::iterator_traits<I>::iterator_category> &&
    std::is_base_of_v<std::output_iterator_tag,
        typename std::iterator_traits<O>::iterator_category>
    ,
    void> // get the type of the enable_if result
input_and_output(I first, I last, O out, int init = 0) {
    *out++ = std::accumulate(first, last, init);
}

int
```

```
main(int, char *[]) {
    input_and_output(
        std::istream_iterator<int>{std::cin},
        std::istream_iterator<int>{},
        std::ostream_iterator<int>{std::cout}
    );
    return 0;
}
```

02-input-and-output/input-and-output-05.cpp

There are two small improvements that still can make the cut.

Let us first have a look at how `std::enable_if` is defined.

```
template< bool B, class T = void >
struct enable_if;
```

Great for us. We could skip the second template parameter because it is already `void`, exactly our return type. Improvement number one is already there.

The second change deals with how we have defined the extra parameter `init` for our `input_and_output` solution.

```
input_and_output(I first, I last, O out, int init = 0) {
```

This could break if we were using a different type for the input and output, like a custom class that had no conversion to/from `int`. Let us use the template facilities just a bit more. The final code is here.

```
#include <iostream> // std:cin/cout
#include <iterator> // std::istream_iterator/ostream_iterator
#include <numeric> // std::accumulate
#include <type_traits> // std::iterator_traits/enable_if/is_base_of

template <typename I, typename O, typename T>
std::enable_if_t<
    // enable if condition
```

```
    std::is_base_of_v<std::input_iterator_tag,
        typename std::iterator_traits<I>::iterator_category> &&
    std::is_base_of_v<std::output_iterator_tag,
        typename std::iterator_traits<O>::iterator_category>
    >
input_and_output(I first, I last, O out, T init = T{}) {
    *out++ = std::accumulate(first, last, init);
}

int
main(int, char *[]) {
    using prob_type = int;
    input_and_output<
        std::istream_iterator<prob_type>,
        std::ostream_iterator<prob_type>,
        prob_type>(
            std::istream_iterator<prob_type>{std::cin},
            std::istream_iterator<prob_type>{},
            std::ostream_iterator<prob_type>{std::cout}
        );
    return 0;
}
```

02-input-and-output/input-and-output-06.cpp

For the sake of a more general approach we even added an alias for `int`, the type used in our solution. All template instantiations use now `prob_type` and with a single change to `using prob_type = int;`, we would change the type everywhere.

But this feels so unnatural. Right before this last piece of code we were free to call `input_output` without any template parameters, because the instantiation was straightforward. We now need to specify the three of them. Unless we always specify a value for `init`, to let the template facilities deduct the type, as it happens in the following code.

```
    input_and_output(
        std::istream_iterator<prob_type>{std::cin},
        std::istream_iterator<prob_type>{},
        std::ostream_iterator<prob_type>{std::cout},
        0
    );
    return 0;
```

```
}
```

02-input-and-output/input-and-output-07.cpp

The problems should be obvious even for the untrained eye (i.e.: the author's)

- First, we are forced now to always provide an argument that we have conceived as having a default value.
- Second, we could pass something else instead of `0` with an incompatible type. So much for our idea to make `init` as generic and default as possible. Or maybe not.

2.2. Summary

Let us cut our losses short, because we have achieved a lot. Having real input in the hand, we have removed the fixed constraints of the challenge and can read any number of values, not just three. Moreover, our STL-like solution has gained type checking for the parameter templates.

But, but, but … "Houston, we have a problem", because our extra parameter is causing problems and the reader has for sure noticed that we have lost our `auto`-magic return type in the solution. The type is now manually determined, when we use `std::enable_if`. Luckily, nothing lasts forever as we will see.

3. Input And Output - Reloaded

Title: Input and Output
Link: https://www.hackerrank.com/challenges/cpp-input-and-output

If the Matrix franchise could reload itself, so can we, improving what we have so far done.

3.1. A Better Input And Output

We ended this challenge with two small problems: the `auto`-return type and the extra `init` parameter that we wanted to be generic, yet compatible with the types the iterators manage.

3.1.1. Deducting Types

Let us first address the second one. Recall that `std::iterator_traits<T>` is the generic interface giving us access to the definitions iterators have to carry. And amongst those definitions we have `value_type`, i.e.: a type. It is the perfect mix: iterators and an API to fetch the type inside the iterator. `init` can now be a lot more generic.

```
#include <iostream> // std:cin/cout
#include <iterator> // std::istream_iterator/ostream_iterator
#include <numeric> // std::accumulate
#include <type_traits> // std::iterator_traits/enable_if/is_base_of

template <typename I, typename O>
std::enable_if_t<
    std::is_base_of_v<std::input_iterator_tag,
        typename std::iterator_traits<I>::iterator_category> &&
    std::is_base_of_v<std::output_iterator_tag,
        typename std::iterator_traits<O>::iterator_category>
    >
input_and_output(I first, I last, O out,
    typename std::iterator_traits<I>::value_type init = {})
{
    *out++ = std::accumulate(first, last, init);
}

int
main(int, char *[]) {
    using prob_type = int;
```

```
    input_and_output(
        std::istream_iterator<prob_type>{std::cin},
        std::istream_iterator<prob_type>{},
        std::ostream_iterator<prob_type>{std::cout}
    );
    return 0;
}
```

03-input-and-output-reloaded/input-and-output-reloaded-01.cpp

This is exactly one of those moments when Italians scream: “Mamma Mia!“. That is really verbose but it works. If we pass something that is not directly convertible to `int`, in our example, there will already be complains in the IDE and compilation will of course fail.

But there is a better way by using a couple made for each other: `decltype` and `std::declval<T>`. Although they belong to different generations, C++11/C++14, they are both jewels. The former lets you deduct the type of the expression passed to it, so one can dynamically calculate, for example a return type or a parameter type. The latter can take an unknown (unevaluated) type `T` to use member functions of it during compilation: i.e.: not going through constructors.

We know that iterators have a `*` operator to retrieve the current value. We can therefore use `std::declval<T>` to access that value by pseudo-constructing `T` and applying the operator.

```
    *std::declval<I>() // Pseudo-constructed T, use the* operator to get a value
```

Once we have a value `decltype` can give us the *declared type*, hence the name.

```
    decltype(*std::declval<I>()) // Get declared type of the "value"
```

It the right moment to apply this to the code in the challenge.

```
#include <iostream> // std:cin/cout
#include <iterator> // std::istream_iterator/ostream_iterator
#include <numeric> // std::accumulate
#include <type_traits> // std::iterator_traits/enable_if/is_base_of
```

```
template <typename I, typename O>
std::enable_if_t<
    std::is_base_of_v<std::input_iterator_tag,
        typename std::iterator_traits<I>::iterator_category> &&
    std::is_base_of_v<std::output_iterator_tag,
        typename std::iterator_traits<O>::iterator_category>
    >
input_and_output(I first, I last, O out, decltype(*std::declval<I>()) init = {}) {
    *out++ = std::accumulate(first, last, init);
}

int
main(int, char *[]) {
    using prob_type = int;
    input_and_output(
        std::istream_iterator<prob_type>{std::cin},
        std::istream_iterator<prob_type>{},
        std::ostream_iterator<prob_type>{std::cout}
    );
    return 0;
}
```

03-input-and-output-reloaded/input-and-output-reloaded-02.cpp

And things look a lot friendlier for the casual code reader and reviewer. Not only that, the errors reported by the IDE and the compilation process are shorter and more amicable. And although both solutions work, we will stick, wherever and whenever possible, with the `declxxx` dynamic duo, because it offers a general interface for deducting types, whereas `std::iterator_traits<T>` is limited to iterators.

3.1.2. Regaining The “auto” Return Type

By using *SFINAE* with `std::enable_if_t` and an *enabler*, i.e.: return type from it, we lost the `auto` facility we were applying to our solution functions. We are forced to manually specify the right type in `std::enable_if_t`.

But see, there is a trick because `std::enable_if_t` returns a type and that means we could put that in the template declaration, where types are allowed. If it fails, the template will not be part of the candidate set. If it succeeds, the template will be a possible match for the template instantiation we look for. Here is how it looks (no changes to `main`)

```
#include <iostream> // std:cin/cout
#include <iterator> // std::istream_iterator/ostream_iterator
#include <numeric> // std::accumulate
#include <type_traits> // std::iterator_traits/enable_if/is_base_of

template <typename I, typename O,
    std::enable_if_t<
        std::is_base_of_v<std::input_iterator_tag,
            typename std::iterator_traits<I>::iterator_category> &&
        std::is_base_of_v<std::output_iterator_tag,
            typename std::iterator_traits<O>::iterator_category>
        >* = nullptr
    >
auto
input_and_output(I first, I last, O out, decltype(*std::declval<I>()) init = {}) {
    *out++ = std::accumulate(first, last, init);
}

int
main(int, char *[]) {
    using prob_type = int;
    input_and_output(
        std::istream_iterator<prob_type>{std::cin},
        std::istream_iterator<prob_type>{},
        std::ostream_iterator<prob_type>{std::cout}
    );
    return 0;
}
```

03-input-and-output-reloaded/input-and-output-reloaded-03.cpp

Recall that the default *enabler* type for `std::enable_if_t` is `void`. And that is what we therefore effectively are left with in the code if the checks are all `true`, is this:

```
template <typename I, typename O, void* = nullptr>
auto
input_and_output(I first, I last, O out, decltype(*std::declval<I>()) init = {})
```

Our third template parameter is valid, of type `void *` and has a default value of `nullptr`. We do not really care about this third parameter because its only purpose is to succeed in

being there or fail, in order to remove itself from the candidate set.

Notice that we are back in business because `auto` is back in business itself, as the return type. This saves us from the burdensome work of having to specify the right type there or as the second parameter to `std::enable_if_t<boolean, type = void>`.

The match candidate will finally look a bit more awkward once the types are in place. Similar to the code below, where we can easily recognize what goes where, how `int` was deduced for `init` and how our `nullptr` enabler is ignored.

```
template<>
void
input_and_output<
    std::istream_iterator<int, ...>, // I
    std::ostream_iterator<int, ...>, // O
    nullptr> // std::enable_if result
(
    std::istream_iterator<int, ...> first,
    std::istream_iterator<int, ...> last,
    std::ostream_iterator<int, ...> out,
    const int &init
)
```

Both small problems are now solved.

3.1.3. SFINAE On Steroids

Steroids is probably an exaggeration. Let us simply say that we can improve the look of the *SFINAE* checks and make things generic for its reusability later somewhere else.

We are indeed checking the nature of our iterators, but whether anyone is willing to type all that every time and actually read it later to understand what is going on, it is another story. Let us then present `std::void_t<class ...>` a C++17 metafunction looking forward to helping us. It transforms valid types passed as templates arguments into `void`. If something is not valid, `std::void_t` will not transform it and this will be the key to remove that candidate.

To profit from `std::void_t` we will use the following technique:

- Use a `struct` that can resolve to `true` if `std::void_t` succeeds in converting all types to

`void`

- Have the same `struct` as a fallback, with default template parameters, that will resolve to `false`. This will be a match if the checks above failed.

To have something that resolves to `true` and `false`, we may use `std::true_type` and `std::false_type`, the perfect *Yin* and *Yang* for the work. Both are specializations of `template<class T, T v> struct integral_constant`, where `T = bool` and each specialization receives the `v = true` and `v = false` values respectively.

For the sake of an example, let us imagine that we want to check if something is simply an iterator with `std::iterator_traits`, disregarding if it is an input, output, forward, or any other type of iterator. Using all the ingredients gathered above, this is how we would do it.

```
template <typename, typename = void>
struct is_it : std::false_type {};

template <typename T>
struct is_it<T, std::void_t<typename std::iterator_traits<T>::iterator_category>>
    : std::true_type {};
```

03-input-and-output-reloaded/input-and-output-reloaded-04.cpp

This is probably easy to understand: the first template candidate is a subclass of `std::false_type` (its value will be `false` when instantiated), whereas the second, a subclass of `std::true_type` that will be `true` when instantiated. The second will be a valid candidate if the template parameter `T` has the iterator trait `iterator_category.` In that case `std::void_t` will convert that to `void`. If not it will fail and our `false_type` derived struct will be the matching candidate.

We could now apply this together with `std::enable_if`

```
template <typename, typename = void>
struct is_it : std::false_type {};

template <typename T>
struct is_it<T, std::void_t<typename std::iterator_traits<T>::iterator_category>>
    : std::true_type {};

template <typename I, typename O,
    std::enable_if_t<is_it<I>{} && is_it<O>{}>* = nullptr>
```

```
auto
input_and_output(I first, I last, O out, decltype(*std::declval<I>()) init = {}) {
    *out++ = std::accumulate(first, last, init);
}
```

03-input-and-output-reloaded/input-and-output-reloaded-04.cpp

It seems like if we have gone from 100 to 0 in microseconds, removing most of the bloat. Notice that we use `{}` to instantiate `is_it<T>` and get the corresponding `true` or `false` value for the logical condition. This is equivalent to having used the `is_xxx_v` form above, a shorthand for `is_xxx::value`. Our `is_it` checker has actually a `::value` (declared as `constexpr static`), because it is a subclass of either `std::false_type` or `std::true_type`.

We can do even better and check multiple parameters in one go by using another C++17 pearl, `std::conjunction<class ...>`. This one takes many types and performs a logical `and` test, returning the value in, you guessed, `::value`. As our types all have a boolean `::value`, the can be used for the logical conjunction.

```
template <typename ...T>
using are_it = std::conjunction<is_it<T>...>;
```

03-input-and-output-reloaded/input-and-output-reloaded-05.cpp

And this leads to an improvement/simplification of the previous code.

Our code can be further simplified.

```
template <typename I, typename O, std::enable_if_t<are_it<I, O>{}>* = nullptr>
auto
input_and_output(I first, I last, O out, decltype(*std::declval<I>()) init = {}) {
    *out++ = std::accumulate(first, last, init);
}
```

03-input-and-output-reloaded/input-and-output-reloaded-05.cpp

The goal, of course, is to check specifically if the iterators are subclasses of *InputIterator* and *OutputIterator*. Let us formulate these traits with `std::void_t` and then pack everything with `using` to make things even cleaner.

First the check for an *InputIterator*.

```
template <typename, typename = void>
struct is_input : std::false_type {};

template <typename I>
struct is_input<I, std::void_t<std::is_base_of<
        std::input_iterator_tag,
        typename std::iterator_traits<I>::iterator_category>>>
    : std::true_type {};
```

03-input-and-output-reloaded/input-and-output-reloaded-06.cpp

Notice that we have used `std::is_base_of` and not the `_v` helper to get the value. `std::void_t` is expecting types and not boolean values.

We can do the same for the output.

```
template <typename, typename = void>
struct is_output : std::false_type {};

template <typename O>
struct is_output<O, std::void_t<std::is_base_of<
        std::input_iterator_tag,
        typename std::iterator_traits<O>::iterator_category>>>
    : std::true_type {};
```

03-input-and-output-reloaded/input-and-output-reloaded-06.cpp

And then check both as shown above with `std::conjunction`.

```
template<typename I, typename O>
using io_iterators = std::conjunction<is_input<I>, is_output<O>>;
```

03-input-and-output-reloaded/input-and-output-reloaded-06.cpp

For bonus points and in order to get the gold medal for packaging and reusability, we can even alias the usage of `std::enable_if_t`.

```
template<typename I, typename O>
using enable_if_io = std::enable_if_t<io_iterators<I, O>{}>*;
```

03-input-and-output-reloaded/input-and-output-reloaded-06.cpp

This means that our definition of `input_output` looks like this now.

```
template <typename I, typename O, enable_if_io<I, O> = nullptr>
auto
input_and_output(I first, I last, O out, decltype(*std::declval<I>()) init = {}) {
    *out++ = std::accumulate(first, last, init);
}
```

03-input-and-output-reloaded/input-and-output-reloaded-06.cpp

Although this looks really nice, we can still make it nicer. Using `decltype(*std::declval<I>())` is for sure a niece piece of work but we could rethink how we use `std::enable_if_t` (or the alias we have created) as a template parameter.

For starters we could alias it, as we have done with the other checks above.

```
template <typename T>
using it_type = decltype(*std::declval<T>());

template <typename I, typename O, enable_if_io<I, O> = nullptr>
auto
input_and_output(I first, I last, O out, it_type<I> init = {}) {
    *out++ = std::accumulate(first, last, init);
}
```

03-input-and-output-reloaded/input-and-output-reloaded-07.cpp

Even nicer. But we have not reworked `std::enable_if_t`. And something nicer would still check the traits but making something useful out of the third template parameter. Now that we can so nicely deduce the type inside iterator with `decltype/std::declval` the spark is there. Let us use that type as the return type in `std::enable_if_t` and then use that as a useful third template parameter instead of having just a dummy `nullptr` placeholder.

```
template<typename I, typename O>
constexpr bool io_iterators_v = io_iterators<I, O>::value;

template <typename T>
using it_type = decltype(*std::declval<T>());
```

```
template<typename I, typename O>
using enable_if_io = std::enable_if_t<io_iterators_v<I, O>, it_type<I>>;

template <typename I, typename O, typename I_Type = enable_if_io<I, O>>
auto
input_and_output(I first, I last, O out, I_Type init = {}) {
    *out++ = std::accumulate(first, last, init);
}
```

03-input-and-output-reloaded/input-and-output-reloaded-08.cpp

We even added a bonus to replicate what we see in the standard our own `_v` helper with `io_iterators_v`. Instead of being a `struct` or a `using` alias, this one is a `constexpr bool`, i.e.: a bool calculated during compile time. That means that we do not have to initialize `io_iterators` or access the `static constexpr value` in it with `::value`, because our helper is doing it for us.

3.1.4. Leaner and Meaner

Everything works, but not really fully. The typename `I_Type` has a default value in the template declaration but we are not specifying the template parameters: the compiler is doing that for us. That means that the real type passed as `I_Type` for the `init` parameter can be something else.

This will be caught by the compiler if the type is not convertible to the type inside the *InputIterator*, but the goal was to invalidate the candidate if the type is wrong.

We can additionally remove some of the bloat we have generated by using `constexpr bool` as we are wrapping `_v` values in `structs` that are subclasses of `true_type` plus a fallback subclass of `false_type`. If the standard `_v` helpers already give us `true` and `false`, let us then use them directly.

```
#include <iostream> // std:cin/cout
#include <iterator> // std::istream_iterator/ostream_iterator
#include <numeric> // std::accumulate
#include <type_traits> // std::iterator_traits/enable_if/is_base_of

template <typename T, typename Tag>
constexpr bool is_it_tag_v = std::is_base_of_v<
    Tag, typename std::iterator_traits<T>::iterator_category>;
```

```
template <typename I>
constexpr bool is_input_v = is_it_tag_v<I, std::input_iterator_tag>;

template <typename O>
constexpr bool is_output_v = is_it_tag_v<O, std::output_iterator_tag>;

template<typename I, typename O>
constexpr bool io_iterators_v = is_input_v<I> && is_output_v<O>;

template <typename T>
using it_type = decltype(*std::declval<T>());

template<typename I, typename O>
using I_Type = std::enable_if_t<io_iterators_v<I, O>, it_type<I>>;
```

03-input-and-output-reloaded/input-and-output-reloaded-09.cpp

There it is. Although `std::void_t`, `std::conjunction` and using the duo `std::false_type` /`true_type` looked good and it actually worked, we now need a lot less and remember the famous axiom: *"less is more"*.

Good old `bool` values that are calculated during compilation, hence `constexpr`, is everything we need. Notice that the type deduction for `init` with `it_type` remains unchanged and we have changed the name of `enable_if_io` to the typename we gave, and will still give, to the parameter `init`, i.e.: we have named the enabler as `I_Type`. See how this is applied in our new definition of `input_and_output`.

```
template <typename I, typename O>
auto
input_and_output(I first, I last, O out, I_Type<I, O> init = {}) {
    *out++ = std::accumulate(first, last, init);
}

int
main(int, char *[]) {
    using prob_type = int;
    input_and_output(
        std::istream_iterator<prob_type>{std::cin},
        std::istream_iterator<prob_type>{},
        std::ostream_iterator<prob_type>{std::cout}
    );
```

```
    return 0;
}
```

03-input-and-output-reloaded/input-and-output-reloaded-09.cpp

It would seem as if we were playing a game of *"Where is the ball?"*. We first had our enabler `std::enable_if_t` in between the template declaration and the function definition. Later we moved it to be the default value of a template parameter (wrapping it in our own `enable_if_io`) And with the name `I_type` it is now the type for the `init` parameter in the function definition.

The `std::enable_if_t` check behind the curtains remains the same and the candidate will be eliminated from the set if either `I` and/or `O` fail the check to see the nature as an *InputIterator* and an *OutputIterator* respectively. What we are doing here is taking advantage of the fact that `std::enable_if_t` returns a type and that type is the one we want to have for `init`.

Recall that type was a hindrance at the beginning, because it would prevent us from using `auto` as the return type. It is now the perfect enforcer for the parameter type. Any wrong type passed to `init` will be caught early, in the definition of our function. In the previous version, any wrong parameter was flagged later, at the `std::accumulate` level.

3.2. Summary

We started with our first ugly and bloated version of *SFINAE* and although we used nice constructs and incredible facilities from the language and the library, this only led to replacing bloat with bloat. We then progressed to make a leaner and meaner version that shall serve us as a model for the future.

4. Hello World - Revolutions

Title: Hello, World!
Link: https://www.hackerrank.com/challenges/cpp-hello-world

After the Matrix was reloaded a revolution followed. Let us try to revolutionize the *"Hello, World!"* challenge.

4.1. SFINAE for the custom iterator

When we modeled our *"Hello, World!"* solution with iterators, including a custom output iterator, we had not yet introduced *SFINAE*. Let us see if our custom iterator can pass the test using the tools we have developed, recovering the `enable_if_io` naming, as we are not using the helper for a type declaration.

```
template <typename T, typename Tag>
constexpr bool is_it_tag_v = std::is_base_of_v<
    Tag, typename std::iterator_traits<T>::iterator_category>;

template <typename I>
constexpr bool is_input_v = is_it_tag_v<I, std::input_iterator_tag>;

template <typename O>
constexpr bool is_output_v = is_it_tag_v<O, std::output_iterator_tag>;

template<typename I, typename O>
constexpr bool io_iterators_v = is_input_v<I> && is_output_v<O>;

template<typename I, typename O>
using enable_if_io = std::enable_if_t<io_iterators_v<I, O>>*;
```

04-hello-world-revolution/hello-world-revolution-01.cpp

Using that arsenal we rework the `hello_world` definition to use `enable_if_io`.

```
template <typename I, typename O, enable_if_io<I, O> = nullptr>
auto
hello_world(I first, I last, O out) {
    return std::copy(first, last, out);
}
```

04-hello-world-revolution/hello-world-revolution-01.cpp

And here comes the surprise, because after crafting that nice, even if verbose, piece of code, the custom output iterator that we crafted before **does not pass** our own check, and we for sure used the right tag. And we are checking with `std::iterator_traits` as we did in the previous chapter to verify the tag that went into the iterator.

```
    using iterator_category = std::output_iterator_tag;
```

04-hello-world-revolution/hello-world-revolution-01.cpp

There will be complaints about `basic_string` (i.e.: `std::string`) not having an `iterator_category` defined. And that simply means that `std::iterator_traits` is trying to tell us that we were lazy.

One has to wonder if we really implemented an output iterator. Before C++17 one could subclass from `std::iterator` and avoid being lazy, because all needed definitions had to be provided. But this was deprecated and a custom iterator needs to have some extra definitions in place to make `std::iterator_traits` happy.

Even more. Our laziness meant we were sloppy and defined the delimiter as `T m_delim`. This does not match what an `std::ostream_iterator` does, because it takes a `const char *`. Let us therefore fix those two things in the custom iterator, presenting the changes.

```
template <typename T>
class os_iterator {
    using ositer = std::ostream_iterator<T>;
    using osdelim = std::ostream_iterator<std::string>;

    ositer m_iter; // wrapped iterator
    osdelim m_itdelim; // wrapped iterator for delimiter
    const std::string m_delim; // separator between elements
    bool m_dodelim = false; // when to start separating

public:
    // needed for an iterator - copy from wrapped iterator
    using iterator_category = typename ositer::iterator_category;
    using value_type = typename ositer::value_type;
    using difference_type = typename ositer::difference_type;
    using pointer = os_iterator *;
```

```
    using reference = os_iterator &;
```

04-hello-world-revolution/hello-world-revolution-02.cpp

The delimiter type issue requires some extra machinery, with a second `std::ostream_iterator` in place. We store the delimiter as an `std::string` and even if in our challenge, the values fed into our custom iterator are also of type `std::string`, they could be anything. And getting our delimiter into that other iterator would fail spectacularly.

To work cleanly we create an alias for each of the `std::ostream_iterator` instances we are hosting, namely: `ositer` for the main type to output and `osdelim` for the delimiter outputting the iterator.

Notice that we copy as many of possible of the iterator definition values from the iterator we are wrapping, namely: `iterator_category`, `value_type` and `difference_type`. But for `pointer` and `reference` the actual values returned by the corresponding methods are returned, i.e.: this iterator.

After this rework, the custom iterator passes all checks and the code compiles. Having been lazy had one advantage: now we know that the *SFINAE* machinery we developed previously works in all cases. We found out as soon as we applied it to the first version of the custom iterator.

4.2. Checking Type Compatibility

In our previous case we had an `init` parameter for which we wanted to set the allowed type in its declaration as a parameter. We achieved this by returning the type gotten via `decltype / std::declval` with `std::enable_if_t`.

When considering the checks for this case, testing if the types of the input iterator can be converted to something accepted by the output iterator seems the logical way to go. That would have been the major price: packing that check into the final alias.

After all, we wrote this previously when we were solving the *"Input and Output"* challenge.

```
    *out++ = *in++ + *in++ + *in++;
```

It would therefore seem logical that the type of `I` is convertible to something accepted by `O`. Once again, `decltype` and `std::declval<T>` come to the rescue, as we can pseudo-construct

objects and use their functionalities.

Let us write a check for that.

```
template <typename I, typename O>
using io_type = decltype(*std::declval<O>() = *std::declval<I>());

template<typename, typename, typename = void>
struct io_i2o : std::false_type {};

template<typename I, typename O>
struct io_i2o<I, O, std::void_t<io_type<I, O>>>
    : std::true_type {};

template<typename I, typename O>
constexpr bool io_i2o_v = io_i2o<I, O>::value;
```

04-hello-world-revolution/hello-world-revolution-03.cpp

We first look for the type of the output, but simulating the input to output operation. In plain English: we are taking the type after the dereferenced declared value of `O` can take the dereferenced value of `I`. Shorthand: `*out = *in` but written as `*std::declval<O>() = *std::declval<I>()` If this works, `decltype` will deliver a type.

To process the type we recover `std::void_t`. Recall that we were previously able to work only with `constexpr bool` as we had values, but we are now deducting a type and have to resort to the `std::false_type / std::true_type` combo of structs we name `io_i2o`, i.e.: “input-output, input to output”.

“Go with the flow”, *“Follow the Path of Least Resistance”*, *“Be Water My Friend”* or any other similar saying is telling us to go with the trend. And the trend favors the use of `_v` helpers. That is why we “degrade” the struct combo down to a `constexpr bool`, as we did previously to have our `io_i2o_v` helper.

The helper is added as a check to `std::enable_if_t` as the second term of a logical and, in which first we check the *input/output* nature of the iterators. If we changed the input `std::array` to hold integers instead of strings, there would be no candidate matching our `hello_world` function request, because the template parameter deduction would fail. One cannot directly put `int` into a `std::string`.

But wait: we are giving the output iterator a manipulated value and the outcome of the

manipulation could be anything, even the instance of a custom class, after we have read some integers. Perfectly valid use cases that we would break by using our `io_i2o_v` as a check. The fact that we get have `const char *` values as the input and output a `std::string` is just a coincidence, it is a specific trait of this problem and not a general check.

We could therefore use it here for the sake of it, but not as a general rule, in we probably want to check if the types of the iterators match our expected definition.

The final code in its entirety is presented now.

```
#include <algorithm> // std::copy
#include <array> // std::array
#include <iostream> // std::cout/cin
#include <iterator> // std::istream/ostream_iterator
#include <type_traits> // std::enable_if/is_base_of

template <typename T>
class os_iterator {
    using ositer = std::ostream_iterator<T>;
    using osdelim = std::ostream_iterator<std::string>;

    ositer m_iter; // wrapped iterator
    osdelim m_itdelim; // wrapped iterator for delimiter
    const std::string m_delim; // separator between elements
    bool m_dodelim = false; // when to start separating

public:
    // needed for an iterator - copy from wrapped iterator
    using iterator_category = typename ositer::iterator_category;
    using value_type = typename ositer::value_type;
    using difference_type = typename ositer::difference_type;
    using pointer = os_iterator *;
    using reference = os_iterator &;

    // constructor replicating the wrapped iterator's constructor
    os_iterator(std::ostream &os, const std::string &delim="") :
        m_iter{os}, m_itdelim{os}, m_delim{delim} {}

    // no-ops because only the assignment (= operator does something)
    auto *operator ->() { return this; } // we wouldn't need this
    auto &operator *() { return *this; }
    auto &operator ++() { return *this; } // ++prefix
```

```
    auto &operator ++(int) { return *this; } // postfix++

    // usual ++ postfix implementation, but we need no tmp
    // because our * and ++ operators are reall no-ops
    // auto operator ++(int) { auto tmp = *this; ++(*this); return tmp; }

    // Operation with the wrapped iterator, choosing when to
    auto &operator =(const T &outval) {
        if(m_dodelim) // output separator before the second/later elements
            *m_itdelim++ = m_delim;
        else
            m_dodelim = true;

        *m_iter++ = outval;
        return *this;
    }
};

template <typename T, typename Tag>
constexpr bool is_it_tag_v = std::is_base_of_v<
    Tag, typename std::iterator_traits<T>::iterator_category>;

template <typename I>
constexpr bool is_input_v = is_it_tag_v<I, std::input_iterator_tag>;

template <typename O>
constexpr bool is_output_v = is_it_tag_v<O, std::output_iterator_tag>;

template<typename I, typename O>
constexpr bool io_iterators_v = is_input_v<I> && is_output_v<O>;

template <typename I, typename O>
using io_type = decltype(*std::declval<O>() = *std::declval<I>());

template<typename, typename, typename = void>
struct io_i2o : std::false_type {};

template<typename I, typename O>
struct io_i2o<I, O, std::void_t<io_type<I, O>>>
    : std::true_type {};

template<typename I, typename O>
constexpr bool io_i2o_v = io_i2o<I, O>::value;
```

```
template<typename I, typename O>
using enable_if_io = std::enable_if_t<io_iterators_v<I, O> && io_i2o_v<I, O>>*;

template <typename I, typename O, enable_if_io<I, O> = nullptr>
auto
hello_world(I first, I last, O out) {
    return std::copy(first, last, out);
}

int
main(int, char *[]) {
    auto hello = std::array{"Hello", "World!"};
    auto out = os_iterator<std::string>{std::cout, ", "};
    hello_world(hello.begin(), hello.end(), out);
    return 0;
}
```

04-hello-world-revolution/hello-world-revolution-03.cpp

4.3. Summary

We had a working solution, but all the work done with out *SFINAE* machinery has paid off, letting us improve our solution to be correct and not only a working solution. C++17 has finally a proper *"Hello, World!"* salutation in place.

5. Types, Types, Types

Title: Basic Data Types
Link: https://www.hackerrank.com/challenges/c-tutorial-basic-data-types/problem

This title probably sounds so bad that even Ballmer's "Developers, Developers, Developers ..." gig would have sold many more vinyl records.

The next challenge gives us a list of basic types. We are expected to read values from the standard input and output the values back to the standard output, with some formatting in a couple of cases. Let us go for it.

5.1. The Obvious Approach

```
#include <iomanip> // std::setprecision
#include <ios> // std::fixed
#include <iostream> // std::cin/cout

int
main(int, char *[]) {
    int i;
    long l;
    char c;
    float f;
    double d;

    std::cin >> i >> l >> c >> f >> d;

    std::cout << i << std::endl;
    std::cout << l << std::endl;
    std::cout << c << std::endl;
    std::cout << std::fixed << std::setprecision(3) << f << std::endl;
    std::cout << std::fixed << std::setprecision(9) << d << std::endl;

    return 0;
}
```

05-basic-data-types/basic-data-types-01.cpp

We simply create some variables of the following types: `int`, `long`, `char`, `float` and `double`, read from `std::cin` and output to `std::cout`. `float` and `double` have some manipulation to

with a fixed number of decimals each, namely 3 and 9.

A simple replacement of this approach with five `std::istream_iterator<T>` and another set of five `std::ostream_iterator<type, "\n">` is for sure not the kind of challenge we are looking for, even if one of the goals is to use iterators here, there and everywhere. Let us therefore skip putting that in written form.

5.2. The Classic Recursive Template

If this were Python, we could iterate over the type names, because those type names are also the classes that we can instantiate to obtain values. But this is not Python and we cannot iterate over the types.

Unless we can. If we consider that one can pass an unlimited number of types to a template and we can then pass those types to another template function, it would seem as if we could iterate. Let us craft a `for_each` looping function. And let us do in the most classic way.

```
template<typename BasicType, typename BasicType2, typename ...BasicTypes>
auto
for_each(size_t fprec = 3, size_t dprec = 9) {
    for_each<BasicType>(fprec, dprec);
    for_each<BasicType2, BasicTypes...>(fprec, dprec);
}
```

05-basic-data-types/basic-data-types-02.cpp

The tricks here should be easy to see, `for_each` takes any number of types by using a template parameter pack, yes, the thing with the `...`. At the same time, and to avoid ambiguity, two of the parameters are specified before the pack. We actually need two, because the pack could be empty and there would be no way to differentiate between these two cases.

```
// 1 template parameter ... the pack is empty
template<typename BasicType, typename ...BasicTypes>

// 1 template parameter
template<typename BasicType> //
```

If the pack is empty, the compiler resolves both cases to be simply a template with one parameter and cannot decide which one will be the match. But if we have two, we are making sure that if the pack is empty, the cases are different.

```
// empty pack, 2 template parameters
template<typename BasicType, typename BasicType2, typename ...BasicTypes>

// 1 template parameter
template<typename BasicType> //
```

That is how the two recursive calls to `for_each` can succeed. The rest will be easy, given that we only need the following.

- A generic `for_each` that instantiates an `std::istream_iterator<BasicType>` and an `std::ostream_iterator<BasicType, "\n">` and copies one element from the former to the latter.
- Because `float` and `double` need some special handling, specializations for those types are needed.
- A call in `main` to start the iteration.

All those ideas expressed in code are shown here.

```
#include <algorithm> // std::copy_n
#include <iomanip> // std::setprecision
#include <ios> // std::fixed
#include <iostream> // std::cin/cout
#include <iterator> // std::ostream_iterator/istream

template<typename BasicType>
auto
for_each(size_t /* fprec */ = 3, size_t /* dprec */ = 9) {
    auto in = std::istream_iterator<BasicType>{std::cin};
    auto out = std::ostream_iterator<BasicType>{std::cout, "\n"};
    std::copy_n(in, 1, out); // *out++ = *in;
}

template<>
auto
for_each<float>(size_t fprec /* = 3 */, size_t /* dprec = 9 */) {
```

```
    const auto def_prec{std::cout.precision()}; // save current prec
    const auto def_flags{std::cout.flags()}; // save current prec
    using BasicType = float;
    std::cout << std::fixed << std::setprecision(fprec);
    auto in = std::istream_iterator<BasicType>{std::cin};
    auto out = std::ostream_iterator<BasicType>{std::cout, "\n"};
    std::copy_n(in, 1, out); // *out++ = *in;
    (std::cout << std::setprecision(def_prec)).flags(def_flags); // reset
}

template<>
auto
for_each<double>(size_t /* fprec = 3 */, size_t dprec /* = 9 */) {
    const auto def_prec{std::cout.precision()}; // save current prec
    const auto def_flags{std::cout.flags()}; // save current prec
    using BasicType = double;
    std::cout << std::fixed << std::setprecision(dprec);
    auto in = std::istream_iterator<BasicType>{std::cin};
    auto out = std::ostream_iterator<BasicType>{std::cout, "\n"};
    std::copy_n(in, 1, out); // *out++ = *in;
    (std::cout << std::setprecision(def_prec)).flags(def_flags); // reset
}

template<typename BasicType, typename BasicType2, typename ...BasicTypes>
auto
for_each(size_t fprec = 3, size_t dprec = 9) {
    for_each<BasicType>(fprec, dprec);
    for_each<BasicType2, BasicTypes...>(fprec, dprec);
}

int
main(int, char *[]) {
    for_each<int, long, char, float, double>();
    return 0;
}
```

05-basic-data-types/basic-data-types-02.cpp

Even if it solves the problem, there are some ugly things. We need to carry around the number of decimal places, `3` and `9`, for `float` and `double`, and to avoid warnings we need to suppress the parameter name. It is not nice.

```
for_each(size_t /* fprec */ = 3, size_t /* dprec */ = 9) {
for_each<float>(size_t fprec /* = 3 */, size_t /* dprec = 9 */) {
for_each<double>(size_t /* fprec = 3 */, size_t dprec /* = 9 */) {
```

05-basic-data-types/basic-data-types-02.cpp

A lot of code is also repeated in the management of the precision for `float` and `double` and in the instantiation of the iterators. Let us address those shortcomings.

5.3. The Classic Template Reworked

```
#include <algorithm> // std::copy_n
#include <iomanip> // std::setprecision
#include <ios> // std::fixed
#include <iostream> // std::cin/cout
#include <iterator> // std::ostream_iterator/istream
#include <utility> // std::make_pair

template <typename BasicType>
auto
in_out() {
    auto in = std::istream_iterator<BasicType>{std::cin};
    auto out = std::ostream_iterator<BasicType>{std::cout, "\n"};
    return std::make_pair(in, out);
}

template<typename BasicType>
auto
for_each(size_t /* fprec */ = 3, size_t /* dprec */ = 9) {
    auto [in, out] = in_out<BasicType>();
    std::copy_n(in, 1, out); // *out++ = *in;
}

template<typename BasicType>
auto
for_each_floating(size_t prec) {
    const auto def_prec{std::cout.precision()}; // save current prec
    const auto def_flags{std::cout.flags()}; // save current prec
    std::cout << std::fixed << std::setprecision(prec);
    auto [in, out] = in_out<BasicType>();
    std::copy_n(in, 1, out); // *out++ = *in;
```

```
    (std::cout << std::setprecision(def_prec)).flags(def_flags); // reset
}

template<>
auto
for_each<float>(size_t fprec /* = 3 */, size_t /* dprec = 9 */) {
    for_each_floating<float>(fprec);
}

template<>
auto
for_each<double>(size_t /* fprec = 3 */, size_t dprec /* = 9 */) {
    for_each_floating<double>(dprec);
}

template<typename BasicType, typename BasicType2, typename ...BasicTypes>
auto
for_each(size_t fprec = 3, size_t dprec = 9) {
    for_each<BasicType>(fprec, dprec);
    for_each<BasicType2, BasicTypes...>(fprec, dprec);
}

int
main(int, char *[]) {
    for_each<int, long, char, float, double>();
    return 0;
}
```

05-basic-data-types/basic-data-types-03.cpp

This is our best try. A function `in_out` serves as a central factory for the `std::istream_iterator` / `std::ostream_iterator` pair we need. And the management of the `float` / `double` precision differences is also outsourced to a `for_each_floating` function.

But that is all. The recursive template nature and the specializations are still there. There has to be something more modern than this.

5.4. The “if constexpr” Express

C++17 introduces a real beast: `if constexpr`, i.e.: we can logically reason during compilation time. It is not that we were not already doing it. The template solutions we presented before were choosing what function to call in the background and with which

types, so the logic was there.

But not being able to implement the logic directly with `if ...` was taking the power out of our hands, and C++17 changes the landscape. Here is the solution after our new white elephant comes into the picture.

```
#include <algorithm> // std::copy_n
#include <iomanip> // std::setprecision
#include <ios> // std::fixed
#include <iostream> // std::cin/cout
#include <iterator> // std::ostream_iterator/istream
#include <type_traits> // std::is_same_v

template<typename BasicType>
auto
for_each(size_t fprec = 3, size_t dprec = 9) {
    const auto def_prec{std::cout.precision()}; // save current prec
    const auto def_flags{std::cout.flags()}; // save current prec

    if constexpr (std::is_same_v<float, BasicType>)
        std::cout << std::fixed << std::setprecision(fprec);
    else if constexpr (std::is_same_v<double, BasicType>)
        std::cout << std::fixed << std::setprecision(dprec);

    auto in = std::istream_iterator<BasicType>{std::cin};
    auto out = std::ostream_iterator<BasicType>{std::cout, "\n"};
    std::copy_n(in, 1, out); // *out++ = *in;
    (std::cout << std::setprecision(def_prec)).flags(def_flags); // reset
}

template<typename BasicType, typename BasicType2, typename ...BasicTypes>
auto
for_each(size_t fprec = 3, size_t dprec = 9) {
    for_each<BasicType>(fprec, dprec);
    for_each<BasicType2, BasicTypes...>(fprec, dprec);
}

int
main(int, char *[]) {
    for_each<int, long, char, float, double>();
    return 0;
}
```

05-basic-data-types/basic-data-types-04.cpp

It really feels like magic. Adding two lines of logic to handle `float` and `double` does remove the need for separate generic and specialized template solutions. And with it the need to duplicate code or to have additional factories and helper functions.

Our, already old and well-known, friend `std::is_same_v` enables the `if constexpr` check. Remember that we discarded `std::is_same` for our iterator checks, because several types (subclasses) of iterators could fit the profile. In this case we want to be specific.

The world is already a better place, but rest assured it can be evaluated from many different perspectives.

5.5. The "if constexpr" Single Method Solution

The introduction of `if constexpr` has allowed a quick removal of specializations and all the bloat (or optimizations) we added. But we still have two template functions. One is doing the work, and the other is simply managing the recursion.

Given the powers granted to, and by, `if constexpr`, it is easy to wonder if the recursion can also be managed by this wonder of wonders. Let us answer that question by directly looking at the following code.

```
#include <algorithm> // std::copy_n
#include <iomanip> // std::setprecision
#include <ios> // std::fixed
#include <iostream> // std::cin/cout
#include <iterator> // std::ostream_iterator/istream
#include <type_traits> // std::is_same_v

template<typename BasicType, typename ...BasicTypes>
auto
for_each(size_t fprec = 3, size_t dprec = 9) {
    const auto def_prec{std::cout.precision()}; // save current prec
    const auto def_flags{std::cout.flags()}; // save current prec

    if constexpr (std::is_same_v<float, BasicType>)
        std::cout << std::fixed << std::setprecision(fprec);
    else if constexpr (std::is_same_v<double, BasicType>)
        std::cout << std::fixed << std::setprecision(dprec);
```

```
    auto in = std::istream_iterator<BasicType>{std::cin};
    auto out = std::ostream_iterator<BasicType>{std::cout, "\n"};
    std::copy_n(in, 1, out); // *out++ = *in;

    (std::cout << std::setprecision(def_prec)).flags(def_flags); // reset

    if constexpr(sizeof...(BasicTypes))
        for_each<BasicTypes...>(fprec, dprec);
}

int
main(int, char *[]) {
    for_each<int, long, char, float, double>();
    return 0;
}
```

05-basic-data-types/basic-data-types-05.cpp

Yes and **no** (a classic German "Jein", i.e.: "Ja" + "Nein"). `if constexpr` alone cannot enter godmode. It needs to work in tandem with a pre-existing member of the C++ family: `sizeof...(pack)`. As the name implies, it allows us to know the number of elements in the pack. Now, we may even perform a logical `if` check on it to know if recursion is needed.

That is the key to remove the trick of having the *"2 parameters + 1 pack"* trick we analyzed before. `for_each` now features a standard *"1 parameter + 1 pack"*. And before recursing, it is checked if the pack has any remaining types. This is really clean, because the *"1 parameter"* has a clear name, `BasicType`, for its usage in the function and `...BasicTypes` contains the remaining types, if any, that would still need to be processed.

5.6. The Fold Expression Miracle

Good things come in pairs, and C++17 did not simply bring `if constexpr`, it also brought *"Fold Expressions"* to the table. A miracle that needs some consideration before one can really understand its depth.

Reading about it and looking at many examples and ideas about how *Fold Expressions* work, one of the approaches seemed like magical. Actually, more magical than the others. It was the application of *Fold Expressions* with the comma operator. In a regular case, the `,` operator works by executing the statements that are separated by the comma and returns the value of the last one. Like in this `while` situation.

```
    int a = 0, b = 100;
    while(a++, a < b)
        do_something();
```

After increasing `a` with `a++`, the boolean value of the comparison `a < b` is what will determine if the `while` loop continues or not. But with a *Fold Expression*, this is different:

```
template<typename ...ParameterPack>
auto
my_function() {
  (do_something<ParameterPack>(), ...);
}
```

Our parameter pack is being folded over the `,` operator, i.e.: each of the members of the pack is being passed one by one to `do_something<ParameterPack>()`. We can for sure work with that. Let us see how.

```
#include <algorithm> // std::copy_n
#include <iomanip> // std::setprecision
#include <ios> // std::fixed
#include <iostream> // std::cin/cout
#include <iterator> // std::ostream_iterator/istream
#include <type_traits> // std::is_same_v

template <typename BasicType>
auto
in_out(size_t fprec = 3, size_t dprec = 9) {
    const auto def_prec{std::cout.precision()}; // save current prec
    const auto def_flags{std::cout.flags()}; // save current prec

    if constexpr (std::is_same_v<float, BasicType>)
        std::cout << std::fixed << std::setprecision(fprec);
    else if constexpr (std::is_same_v<double, BasicType>)
        std::cout << std::fixed << std::setprecision(dprec);

    auto in = std::istream_iterator<BasicType>{std::cin};
    auto out = std::ostream_iterator<BasicType>{std::cout, "\n"};
    std::copy_n(in, 1, out); // *out++ = *in;
```

```
    (std::cout << std::setprecision(def_prec)).flags(def_flags); // reset
};

template<typename ...BasicTypes>
auto
for_each(size_t fprec = 3, size_t dprec = 9) {
    (in_out<BasicTypes>(fprec, dprec), ...);
}

int
main(int, char *[]) {
    for_each<int, long, char, float, double>();
    return 0;
}
```

05-basic-data-types/basic-data-types-06.cpp

Recursion is managed by the *Fold Expression* with the `,` operator. We have seen ourselves in the need to outsource the processing of the type again to the `in_out` function. And we are in the business of less is more.

5.7. The Fold Expression Single Method

This is where another modern member of the C++ family comes to the rescue, because we can make good use of a *lambda expression*, i.e.: an unnamed function declared there where you need it. We can move the content of `in_out` inside our `for_each` to have it working together with the *fold expression*.

But some reworking will be needed, because a *lambda expression* cannot use templates. OK, because we live in the future we know this is possible with C++20, but our compiler is a veteran still using C++17. Luckily, not all hope is lost, because we can invite a well-known friend to the party: `decltype`. This time it comes without the famous partner `std::declval<T>`, but this time extra help in deducing the type is not needed, because it is partnering with someone else: `auto`. In this case as a parameter type, for the first time ever … in our C++ ramblings, of course.

Because a template parameter is not possible for the *lambda*, we have crafted an automatic parameter.

```
    auto in_out = [&fprec, &dprec](auto x) {
```

05-basic-data-types/basic-data-types-07.cpp

The parameter will `auto`-magically adapt itself to what we pass. It is not that we have invented this, because it was invented with C++14, with the concept being named a *generic lambda*.

Notice how we also have `fprec` and `dprec` in the *lambda expression*. They are captures, hence the lack of a default value, because it is captured from the surroundings to be used inside the *lambda*.

To make our *fold expression* cooperate with the lambda, we need to make a small change. In the previous example, we were invoking `in_out<BasicTypes>()`, because we had a templated function. Not being able to use templated *lambda expressions*, the best we can do is to pass a value of the type, by instantiating the type.

```
    (in_out(BasicTypes{}), ...);
```

05-basic-data-types/basic-data-types-07.cpp

A value that can be later turned back, for our purposes, into a type with `decltype`.

```
        using BasicType = decltype(x);  // deduct type of dummy x value
```

05-basic-data-types/basic-data-types-07.cpp

And here is the complete single method approach with a *folding expression*.

Here is the single method, thanks to the cooperation of all those heroes.

```
#include <algorithm> // std::copy_n
#include <iomanip> // std::setprecision
#include <ios> // std::fixed
#include <iostream> // std::cin/cout
#include <iterator> // std::ostream_iterator/istream
#include <type_traits> // std::is_same_v

template<typename ...BasicTypes>
```

```
auto
for_each(size_t fprec = 3, size_t dprec = 9) {
    auto in_out = [&fprec, &dprec](auto x) {
        const auto def_prec{std::cout.precision()}; // save current prec
        const auto def_flags{std::cout.flags()}; // save current flags

        using BasicType = decltype(x);  // deduct type of dummy x value
        if constexpr (std::is_same_v<float, BasicType>)
            std::cout << std::fixed << std::setprecision(fprec);
        else if constexpr (std::is_same_v<double, BasicType>)
            std::cout << std::fixed << std::setprecision(dprec);

        auto in = std::istream_iterator<BasicType>{std::cin};
        auto out = std::ostream_iterator<BasicType>{std::cout, "\n"};
        std::copy_n(in, 1, out); // *out++ = *in;

        (std::cout << std::setprecision(def_prec)).flags(def_flags); // reset
    };

    (in_out(BasicTypes{}), ...);
}

int
main(int, char *[]) {
    for_each<int, long, char, float, double>();
    return 0;
}
```

05-basic-data-types/basic-data-types-07.cpp

There is something worth mentioning before we carry on. Because the *fold expression* is managing the iteration of the parameter pack, this parameter pack is the only parameter to the template. No tricks with a first parameter or even two parameters to differentiate the candidates to be matched. A real improvement.

```
template<typename ...BasicTypes>
```

05-basic-data-types/basic-data-types-07.cpp

5.8. The Last Crown Jewel - std::variant

We need nothing more until we realize we do, because C++17 just keeps on giving. It introduced a type-safe union that can hold any value. One at a time, of course. One only needs to specify the types that may be inside `std::variant` as template parameters.

It would seem as if someone was thinking about our little problem here when they designed this functionality, because that someone also added an `std::variant_alternative<size_t, MyVariant>`. I.e.: we can retrieve the type specified at a given position for an `std::variant`.

Somehow we need to pass a `size_t` to our template function and check if we have come to the last type or else recurse. The pieces of the puzzle are all there:

- templates do not only take types, they also take values.
- `if constexpr` has already helped us with the logic to control the recursion.

With all the accumulated experience from the previous attempts, we can do this in one go.

```
#include <algorithm> // std::copy_n
#include <iomanip> // std::setprecision
#include <ios> // std::fixed
#include <iostream> // std::cin/cout
#include <iterator> // std::ostream_iterator/istream
#include <type_traits> // std::is_same_v
#include <variant> // std::variant

template <typename Variant, size_t i = 0>
auto
for_each(size_t fprec = 3, size_t dprec = 9) {
    const auto def_prec{std::cout.precision()}; // save current prec
    const auto def_flags{std::cout.flags()}; // save current prec

    using BasicType = std::variant_alternative_t<i, Variant>; // get type

    if constexpr (std::is_same_v<float, BasicType>)
        std::cout << std::fixed << std::setprecision(fprec);
    else if constexpr (std::is_same_v<double, BasicType>)
        std::cout << std::fixed << std::setprecision(dprec);

    auto in = std::istream_iterator<BasicType>{std::cin};
```

```
    auto out = std::ostream_iterator<BasicType>{std::cout, "\n"};
    std::copy_n(in, 1, out); // *out++ = *in;

    (std::cout << std::setprecision(def_prec)).flags(def_flags); // reset

    if constexpr ((i + 1) < std::variant_size_v<Variant>)
        for_each<Variant, i + 1>(fprec, dprec);
}

int
main(int, char *[]) {
    using BasicTypes = std::variant<int, long, char, float, double>;
    for_each<BasicTypes>();
    return 0;
}
```

05-basic-data-types/basic-data-types-08.cpp

Our looping `for_each` now takes only one template parameter for the types, the `std::variant` that carries those types. It takes a second parameter `size_t i = 0`, to let us know the current type to be retrieved.

```
    using BasicType = std::variant_alternative_t<i, Variant>; // get type
```

05-basic-data-types/basic-data-types-08.cpp

And retrieved it is. That is the key to having things working. Notice how we have to use a strange notation with 'i'.

```
    if constexpr ((i + 1) < std::variant_size_v<Variant>)
```

05-basic-data-types/basic-data-types-08.cpp

Being `size_t i` a template parameter, it is “read-only” for the function and we cannot simply do `i++`, because it is not an *lvalue*, hence the need to use `i + 1` for the recursive call.

5.9. Summary

We have not iterated over types in a Python-like style. But we did it, not only with a classic recursive template approach. We have added as much as possible from the weaponry

provided by C++17 to let us have single method approaches that control the recursion internally and reduced the template parameters to manage all types with a single one.

It is time to bury this topic, although there may be more to come.

6. Types - Resurrected

Title: Basic Data Types
Link: https://www.hackerrank.com/challenges/c-tutorial-basic-data-types/problem

6.1. Iterating Iterators

That oxymoron is simply there to remark that for the *"Basic Data Types"* challenge we actually iterated over the types and we did not see a single iterator along the ways. In spite of all the wonders we applied, `if constexpr`, `sizeof...`, *fold expressions*, and `std::variant`, we were working in compile-time mode. There was no room for iterators.

But there is always room for improvement, and we are going to find room for the iterators.

6.2. Templated Iterators

Obviously, if we have been working with templates to address the problem of moving around a list of types, templated iterators seem like the ways to go.

We are going to use two new C++ elements, and as it has been the case, we are going to couple another C++17 present with C++ veterans. The modern element is `std::any`, a class capable of holding any value, that has to be retrieved with the proper type, using `std::any_cast<T>(any_instance)`. Our veterans will be `std::vector`, to store factories of iterators for different types. Those factories will be hosted by a C++11 family member, namely `std::function`.

Recall that we previously had different ways of iterating over the list of types. To build something more complex we still need a first iteration over them. We will be building upon the latest solution, using `std::variant` and looping with a `size_t i` template parameter.

Let us see how the factories and the storage for them are modeled.

```
using InTypeProcs = std::vector<std::function<std::any(std::istream &)>>;
include::[]
using OutTypeProcs = std::vector<
    std::function<void(std::ostream &, const std::any &, size_t, size_t)>>;
```

basic-data-types-resurrections-01.cpp

For the input, there will be a vector holding functions that return `std::any` and take an `std::istream &` as the only parameter. The `std::vector` for the output will hold functions that take an `std::ostream &`, the value in the form of `std::any`, and finally two `size_t` parameters. We need those two to pass the values controlling the decimal places for `float` and `double`.

Let us see how we will use that for the input iterator, which we name `vistream_iterator` (where `v` stands for `variant`).

```
template <typename Variant, size_t i = 0>
auto
for_each_in(InTypeProcs &intypeprocs) {
    auto intypeproc = [](std::istream &is) {
        using BasicType = std::variant_alternative_t<i, Variant>; // get type
        auto in = std::istream_iterator<BasicType>{is};
        return std::any{*in};
    };
    intypeprocs.push_back(intypeproc);

    if constexpr ((i + 1) < std::variant_size_v<Variant>)
        for_each_in<Variant, i + 1>(intypeprocs);
}
include::[]
include::[]
    vistream_iterator(std::istream &is)
        : m_is{is}, m_end{not std::variant_size_v<Variant>} {

        for_each_in<Variant>(m_intypeprocs);
    }
```

basic-data-types-resurrections-01.cpp

Just a we did previously, we recurse over the types stored in the `std::variant` we receive as a template parameter. For each type a *lambda* is created and stored in a vector. That *lambda* will later take over and create an `std::istream_iterator` of the corresponding type, returning it embedded in an `std::any` instance.

The key to using those *lambda* expressions is in the `*` and `++` operators.

```
    auto operator *() const {
        auto intypeproc = *std::next(m_intypeprocs.begin(), m_pos);
        return intypeproc(m_is);
    }
    // Prefix increment
    auto operator ++() {
        m_end = (++m_pos == std::variant_size_v<Variant>);
        return *this;
    }
```

06-basic-data-types-resurrections/basic-data-types-resurrections-01.cpp

When someone invokes the `*` operator, the next lambda is retrieved. Literally, `std::next` is always called with an offset to the beginning of the container. That offset is incremented during `++` operations, checking if the end of the container has been reached.

There is of course a corresponding recursion procedure for the output.

```
template <typename Variant, size_t i = 0>
auto
for_each_out(OutTypeProcs &otypeprocs) {
    auto proc = [](std::ostream &os, const std::any &val,
                   size_t fprec = 3, size_t dprec = 9) {

        using BasicType = std::variant_alternative_t<i, Variant>; // get type
        const auto def_prec{os.precision()}; // save current prec
        const auto def_flags{os.flags()}; // save current prec
        auto out = std::ostream_iterator<BasicType>{os};
        if constexpr (std::is_same_v<float, BasicType>)
            os << std::fixed << std::setprecision(fprec);
        else if constexpr (std::is_same_v<double, BasicType>)
            os << std::fixed << std::setprecision(dprec);

        *out++ = std::any_cast<const BasicType &>(val);
        (os << std::setprecision(def_prec)).flags(def_flags); // reset
    };
    otypeprocs.push_back(proc);

    if constexpr ((i + 1) < std::variant_size_v<Variant>)
        for_each_out<Variant, i + 1>(otypeprocs);
}
```

06-basic-data-types-resurrections/basic-data-types-resurrections-01.cpp

The code in the *lambda* does the same as we did previously. The only novelty, as is the case with the input, is the pre-creation of the *lambdas* that go into the `std::vector`. The `vostream_iterator` in which we have our storage, will later retrieve the corresponding *lambda*.

```
    auto &operator =(const std::any &val) const {
        auto otypeproc = *std::next(m_otypeprocs.begin(), m_pos);
        otypeproc(m_os, val, m_fprec, m_dprec);
        m_os << m_delim;
        return *this;
    }
```

06-basic-data-types-resurrections/basic-data-types-resurrections-01.cpp

As was the case with our previous custom output iterator the real work takes place in the `=` operators. Here also, `std::next` is used to fetch the *lambda* that knows what to do with the `std::any`, given also an output stream and the values for the decimal places for `float` and `double`.

It is now possible to create an STL-like solution. Here is the code for the entire solution.

```
#include <any> // std::any
#include <algorithm> // std::copy_n
#include <functional> // std::function
#include <iomanip> // std::setprecision
#include <ios> // std::fixed
#include <iostream> // std::cin/cout
#include <iterator> // std::ostream_iterator/istream
#include <type_traits> // std::is_same_v
#include <variant> // std::variant
#include <vector> // std::vector

using InTypeProcs = std::vector<std::function<std::any(std::istream &)>>;

template <typename Variant, size_t i = 0>
auto
for_each_in(InTypeProcs &intypeprocs) {
    auto intypeproc = [](std::istream &is) {
        using BasicType = std::variant_alternative_t<i, Variant>; // get type
```

```
        auto in = std::istream_iterator<BasicType>{is};
        return std::any{*in};
    };
    intypeprocs.push_back(intypeproc);

    if constexpr ((i + 1) < std::variant_size_v<Variant>)
        for_each_in<Variant, i + 1>(intypeprocs);
}

template <typename Variant>
struct vistream_iterator {
    // Iterator tags
    using iterator_category = std::input_iterator_tag;
    using difference_type   = std::ptrdiff_t;
    using value_type        = std::any;
    using reference         = value_type &;
    using pointer           = value_type *;

    InTypeProcs m_intypeprocs;
    size_t m_pos = 0;

    std::istream &m_is;
    bool m_end = false;

    vistream_iterator(std::istream &is)
        : m_is{is}, m_end{not std::variant_size_v<Variant>} {

        for_each_in<Variant>(m_intypeprocs);
    }
    // dummy m_is initialization to avoid using -fpermissive
    vistream_iterator() : m_is{std::cin}, m_end{true} {}

    auto operator ->() const noexcept { return this; }

    auto operator *() const {
        auto intypeproc = *std::next(m_intypeprocs.begin(), m_pos);
        return intypeproc(m_is);
    }
    // Prefix increment
    auto operator ++() {
        m_end = (++m_pos == std::variant_size_v<Variant>);
        return *this;
    }
    // Postfix increment
```

```
    auto operator ++(int) const {
        auto tmp = *this;
        ++(*this);
        return tmp;
    }

    auto operator ==(const vistream_iterator& other) const {
        return m_end ? other.m_end : (other.m_end ? false : m_pos == other.m_pos);
    };
    auto operator !=(const vistream_iterator& other) const {
        return not (*this == other);
    }
};

//////////////////////////////////////////////////////////////////////////
using OutTypeProcs = std::vector<
    std::function<void(std::ostream &, const std::any &, size_t, size_t)>>;

template <typename Variant, size_t i = 0>
auto
for_each_out(OutTypeProcs &otypeprocs) {
    auto proc = [](std::ostream &os, const std::any &val,
                   size_t fprec = 3, size_t dprec = 9) {

        using BasicType = std::variant_alternative_t<i, Variant>; // get type
        const auto def_prec{os.precision()}; // save current prec
        const auto def_flags{os.flags()}; // save current prec
        auto out = std::ostream_iterator<BasicType>{os};
        if constexpr (std::is_same_v<float, BasicType>)
            os << std::fixed << std::setprecision(fprec);
        else if constexpr (std::is_same_v<double, BasicType>)
            os << std::fixed << std::setprecision(dprec);

        *out++ = std::any_cast<const BasicType &>(val);
        (os << std::setprecision(def_prec)).flags(def_flags); // reset
    };
    otypeprocs.push_back(proc);

    if constexpr ((i + 1) < std::variant_size_v<Variant>)
        for_each_out<Variant, i + 1>(otypeprocs);
}

template <typename Variant>
class vostream_iterator {
```

```
public:
    // needed for an iterator
    using iterator_category = std::output_iterator_tag;
    using difference_type   = std::ptrdiff_t;
    using value_type        = std::any;
    using reference         = value_type &;
    using pointer           = value_type *;

private:
    OutTypeProcs m_otypeprocs;
    size_t m_pos = 0;

    std::ostream &m_os;
    std::string m_delim;
    size_t m_fprec;
    size_t m_dprec;

public:
    // constructor replicating the wrapped iterator's constructor
    vostream_iterator(std::ostream &os, const std::string &delim,
        size_t fprec = 3, size_t dprec = 9)
        : m_os{os}, m_delim(delim), m_fprec(fprec), m_dprec(dprec) {

        for_each_out<Variant>(m_otypeprocs);
    }
    // no-ops because only the assignment (= operator does something)
    auto operator ->() const { return this; };
    auto &operator *() const { return *this; }
    auto &operator ++() { ++m_pos; return *this; } // ++prefix
    auto operator ++(int) const { //postfix++
        auto tmp = *this;
        ++(*this);
        return tmp;
    }
    // Operation with the wrapped iterator, choosing when to output the sep
    auto &operator =(const std::any &val) const {
        auto otypeproc = *std::next(m_otypeprocs.begin(), m_pos);
        otypeproc(m_os, val, m_fprec, m_dprec);
        m_os << m_delim;
        return *this;
    }
};

// Solution
```

```
template<typename I, typename O>
auto
basic_types(I first, I last, O out) {
    return std::copy(first, last, out);
}

int
main(int, char *[]) {
    using BasicTypes = std::variant<int, long, char, float, double>;
    basic_types(
        vistream_iterator<BasicTypes>{std::cin},
        vistream_iterator<BasicTypes>{},
        vostream_iterator<BasicTypes>{std::cout, "\n"}
    );
    return 0;
}
```

06-basic-data-types-resurrections/basic-data-types-resurrections-01.cpp

6.3. Criticism

Even if we have made our code dynamic and STL-like, we have probably made things overly complex. Especially because there is an interdependency between the input and the output as they have to remain synchronized to ensure the output iterator retrieves the proper *lambda* to match the type stored in the `std::any` instance.

There must be a way to remove that link between the input and output iterators.

6.4. Dynamic Type Output

Tackling the nature of the input iterator is not going to be easy because it has to respect the ordering of the target types. But for starters, we can do sensible things such as taking the generation of the input functions out of the constructor.

In the case of the output iterator we can go several steps beyond also doing that. `std::any` has a very interesting member function named `type`. As the name indicates it lets us know things about the type being held because it *"returns the typeid of the contained value"*. That means we get the following done for us: `typeid(type_in_std::any)`, returning the type information in an instance of `std::type_info`.

C++11 brought us `std::type_index`, a wrapper around the latter to make it possible to use

the type information as the key in mapping containers. And that is what is going to help us with a simplification of the output iterator. We first put the output functions in an `std::map` using an `std::type_index` as the key. When the time comes to do actual output, `std::any`, via `type()`, gives us a `type_info`, that we can wrap inside an `std::type_index` to look for the proper function in the `std::map`.

```
using BasicTypes = std::variant<int, long, char, float, double>;

using InTypeProc = std::function<std::any(std::istream &)>;
using InTypeProcs = std::vector<InTypeProc>;

using OutTypeProc = std::function<void(std::ostream &, const std::any &)>;
using OutTypeProcs = std::map<std::type_index, OutTypeProc>;

static InTypeProcs intypeprocs; // storage for the input fetching procs
static OutTypeProcs outtypeprocs;  // storage for the output procs

template <typename Variant, size_t i = 0>
auto
for_each(InTypeProcs &itp, OutTypeProcs &otp) {
    using BasicType = std::variant_alternative_t<i, Variant>; // get type
    auto intypeproc = [](std::istream &is) {
        return std::any{*std::istream_iterator<BasicType>{is}};
    };
    itp.push_back(intypeproc);

    auto outtypeproc = [](std::ostream &os, const std::any &a) {
        os << std::any_cast<BasicType>(a);
    };
    auto typeidx = std::type_index(typeid(BasicType));
    otp.insert(std::make_pair(typeidx, outtypeproc));

    if constexpr ((i + 1) < std::variant_size_v<Variant>)
        for_each<Variant, i + 1>(itp, otp);
}
```

06-basic-data-types-resurrections/basic-data-types-resurrections-02.cpp

We have gone the `using` way to have a clear view of the containers holding our input/output functions, the ones working directly with `std::any`. Iterating over the types of the `std::variant` used to hold the types, both the `std::vector` and the `std::map` are generated.

The input iterator does not change much because it still needs to know the number of types it has to fetch and it still keeps an index with `m_pos`.

On the other hand `vostream_iterator` is no longer a template based class.

```
////////////////////////////////////////////////////////////////////////////
class vostream_iterator {
public:
```

06-basic-data-types-resurrections/basic-data-types-resurrections-02.cpp

That also means there is no index to track in parallel to the index of the input iterator. We can now have a generic approach to finding the right function by retrieving it from the map with the information from the `std::any` instance.

```
    auto &operator =(const std::any &a) const {
        const auto def_prec{m_os.precision()}; // save current prec
        const auto def_flags{m_os.flags()}; // save current flags

        if (a.type() == typeid(float))
            m_os << std::fixed << std::setprecision(m_fprec);
        else if (a.type() == typeid(double))
            m_os << std::fixed << std::setprecision(m_dprec);

        // use structured bindings to get our out-type-proc
        auto &[_, otproc] = *outtypeprocs.find(std::type_index(a.type()));
        otproc(m_os, a);

        (m_os << std::setprecision(def_prec)).flags(def_flags); // reset
        m_os << m_delim;
        return *this;
    }
```

06-basic-data-types-resurrections/basic-data-types-resurrections-02.cpp

For bonus points, we also use `typeid(type)` to find out if the fixed point precision has to be set for `float` and `double`. We were previously doing that directly in the output procedures with `if constexpr`, but can do it dynamically here now thanks to the `type()` member function of `std::any`.

To make our life easier we use another of the goodies of C++17, i.e.: *"Structured Bindings"*.

It feels almost like unpacking an iterable in Python.

```
        // use structured bindings to get our out-type-proc
        auto &[_, otproc] = *outtypeprocs.find(std::type_index(a.type()));
        otproc(m_os, a);
```

06-basic-data-types-resurrections/basic-data-types-resurrections-02.cpp

It also feels like magic, because we can unpack the `std::pair` held by the map directly into variables, that can be declared as `auto`. Just like in Python we use `_` as a placeholder for the variable we want to ignore. This will apparently be official in C++26, when one will be able to use officially use the placeholder and do so as many times as needed.

Let us see the complete code of this second solution.

```
#include <any> // std::any
#include <algorithm> // std::copy_n
#include <functional> // std::function
#include <iomanip> // std::setprecision
#include <ios> // std::fixed
#include <iostream> // std::cin/cout
#include <iterator> // std::ostream_iterator/istream
#include <map> // std::map
#include <type_traits> // std::is_same_v
#include <variant> // std::variant
#include <vector> // std::vector
#include <typeindex> // std::type_index

using BasicTypes = std::variant<int, long, char, float, double>;

using InTypeProc = std::function<std::any(std::istream &)>;
using InTypeProcs = std::vector<InTypeProc>;

using OutTypeProc = std::function<void(std::ostream &, const std::any &)>;
using OutTypeProcs = std::map<std::type_index, OutTypeProc>;

static InTypeProcs intypeprocs; // storage for the input fetching procs
static OutTypeProcs outtypeprocs;  // storage for the output procs

template <typename Variant, size_t i = 0>
auto
```

```
for_each(InTypeProcs &itp, OutTypeProcs &otp) {
    using BasicType = std::variant_alternative_t<i, Variant>; // get type
    auto intypeproc = [](std::istream &is) {
        return std::any{*std::istream_iterator<BasicType>{is}};
    };
    itp.push_back(intypeproc);

    auto outtypeproc = [](std::ostream &os, const std::any &a) {
        os << std::any_cast<BasicType>(a);
    };
    auto typeidx = std::type_index(typeid(BasicType));
    otp.insert(std::make_pair(typeidx, outtypeproc));

    if constexpr ((i + 1) < std::variant_size_v<Variant>)
        for_each<Variant, i + 1>(itp, otp);
}

template <typename Variant>
struct vistream_iterator {
    // Iterator tags
    using iterator_category = std::input_iterator_tag;
    using difference_type   = std::ptrdiff_t;
    using value_type        = std::any;
    using reference         = value_type &;
    using pointer           = value_type *;

    size_t m_pos = 0;

    std::istream &m_is = std::cin;
    bool m_end = true;

    vistream_iterator(std::istream &is) :
        m_is{is}, m_end{not std::variant_size_v<Variant>} {}
    vistream_iterator() {}

    auto operator ->() const noexcept { return this; }
    auto operator *() const {
        auto intypeproc = *std::next(intypeprocs.begin(), m_pos);
        return intypeproc(m_is);
    }
    // Prefix increment
    auto operator ++() {
        m_end = (++m_pos == std::variant_size_v<Variant>);
        return *this;
```

```
    }
    // Postfix increment
    auto operator ++(int) const {
        auto tmp = *this;
        ++(*this);
        return tmp;
    }

    auto operator ==(const vistream_iterator& other) const {
        return m_end ? other.m_end : (other.m_end ? false : m_pos == other.m_pos);
    };
    auto operator !=(const vistream_iterator& other) const {
        return not (*this == other);
    }
};

////////////////////////////////////////////////////////////////////////////
class vostream_iterator {
public:
    // needed for an iterator
    using iterator_category = std::output_iterator_tag;
    using difference_type   = std::ptrdiff_t;
    using value_type        = std::any;
    using reference         = value_type &;
    using pointer           = value_type *;

private:
    std::ostream &m_os;
    std::string m_delim;
    size_t m_fprec;
    size_t m_dprec;

public:
    // constructor replicating the wrapped iterator's constructor
    vostream_iterator(std::ostream &os, const std::string &delim,
        size_t fprec = 3, size_t dprec = 9)
        : m_os{os}, m_delim{delim}, m_fprec{fprec}, m_dprec{dprec} {}
    // no-ops because only the assignment (= operator does something)
    auto operator ->() const { return this; };
    auto &operator *() const { return *this; }
    auto &operator ++() { return *this; } // ++prefix
    auto operator ++(int) const { return *this; }

    auto &operator =(const std::any &a) const {
```

```
        const auto def_prec{m_os.precision()}; // save current prec
        const auto def_flags{m_os.flags()}; // save current flags

        if (a.type() == typeid(float))
            m_os << std::fixed << std::setprecision(m_fprec);
        else if (a.type() == typeid(double))
            m_os << std::fixed << std::setprecision(m_dprec);

        // use structured bindings to get our out-type-proc
        auto &[_, otproc] = *outtypeprocs.find(std::type_index(a.type()));
        otproc(m_os, a);

        (m_os << std::setprecision(def_prec)).flags(def_flags); // reset
        m_os << m_delim;
        return *this;
    }
};

// Solution
template<typename I, typename O>
auto
basic_types(I first, I last, O out) {
    return std::copy(first, last, out);
}

int
main(int, char *[]) {
    for_each<BasicTypes>(intypeprocs, outtypeprocs);
    basic_types(
        vistream_iterator<BasicTypes>{std::cin},
        vistream_iterator<BasicTypes>{},
        vostream_iterator{std::cout, "\n"}
    );
    return 0;
}
```

06-basic-data-types-resurrections/basic-data-types-resurrections-02.cpp

6.5. Going The Iterating Mile

We started this new chapter because iterating over the list of types was not the goal. That is why we now have solutions based on iterators. Unfortunately, these solutions rely on knowledge of the `std::variant` and the containers with helpers to fetch types from

`std::cin` and put them to `std::out`.

A real success would be to remove those two dependencies and make the code even more generic. And yes, we can!

Let us start by backpedaling a bit. We decided to go for `std::any` as the vehicle to pass instances of the types back and forth, because it can hold anything. However, that flexibility comes with a price: getting the value out of an `std::any` instance is hard, hence the need to have so many *lambda expressions*, why we synchronized the iterators and had to work with `std::type_info`.

Let us look at three ideas to streamline our solution

1. Given that we are using `std::variant` to convey the types and it can also hold instances of the types, it may be a lot easier to work with it.
2. Let our iterator, literally iterate over the container, instead of doing something with the container. I.e.: our custom iterator will work with iterators to traverse the container of *lambda expressions*.
3. If getting the value out of an `std::variant`, with the right type, is easier than with `std::any`, we may remove the *lambda expressions* for the output and make the output iterator really generic.

The code generating the *lambdas* for fetching the values from `std::cin` does not change. It is still an `std::vector`, holding *lambdas*, but this time the return value is an `std::variant`. Let us show it.

```
using BasicTypes = std::variant<int, long, char, float, double>;
using InTypeProc = std::function<BasicTypes(std::istream &)>;
using InTypeProcs = std::vector<InTypeProc>;

static InTypeProcs intypeprocs;

template <typename Variant, size_t i = 0>
auto
for_each_in(InTypeProcs &itp) {
    using BasicType = std::variant_alternative_t<i, Variant>; // get type
    auto intypeproc = [](std::istream &is) {
        return Variant{*std::istream_iterator<BasicType>{is}};
    };
```

06-basic-data-types-resurrections/basic-data-types-resurrections-03.cpp

No *lambdas* are generated for the output.

The input iterator works with a `typename I`, that must be an iterator.

```
template <typename I>
struct vistream_iterator {
    // Iterator tags
    using iterator_category = std::input_iterator_tag;
    using difference_type   = std::ptrdiff_t;
    using value_type        = std::any;
    using reference         = value_type &;
    using pointer           = value_type *;

    std::istream &m_is = std::cin;
    I m_first, m_last;
    bool m_end = true;

    vistream_iterator(std::istream &is, I first, I last)
        : m_is{is}, m_first{first}, m_last{last}, m_end{first == last} {}
    vistream_iterator(I last) : m_first{last}, m_last{last} {}

    auto operator ->() const noexcept { return this; }
    auto operator *() const {
        return (*m_first)(m_is); // fetched intypeproc delivers result
    }
    // Prefix increment
    auto &operator ++() {
        m_end = ((++m_first) == m_last);
        return *this;
    }
    // Postfix increment
    auto operator ++(int) const {
        auto tmp = *this;
        ++(*this);
        return tmp;
    }
```

06-basic-data-types-resurrections/basic-data-types-resurrections-03.cpp

It is used during construction to get the beginning, with `first`, and the end, with `last`, of

the range that holds procedures to fetch the types. Our input iterator no longer needs to know things about the container. Because the range is not directly controlled by our iterator, the second constructor , defining the end of iteration, also needs to know where the end of the real range is. And that it is why it takes also a `last` parameter.

Granted, with some *SFINAE* we could restrict and check that the iterator `I` is of type *InputIterator* and delivers the expected result when dereferenced.

Things are even better for the output iterator. We had already decoupled it from the `std::variant` but it still had to rely on a set of external functions (*lambdas*) to output each type, as the type had to be forcefully extracted from `std::any`. With the use of `std::variant` things are a lot cleaner.

```
template<typename V>
auto &operator =(const V &v) const {
    const auto def_prec{m_os.precision()}; // save current prec
    const auto def_flags{m_os.flags()}; // save current flags

    // visit our variant val and get the type in auto format
    std::visit([&](auto &arg) {  // arg is the value inside variant v
        using T = std::decay_t<decltype(arg)>; // deduce actual type
        // do something if float or double
        if constexpr (std::is_same_v<T, float>)
            m_os << std::fixed << std::setprecision(m_fprec);
        else if constexpr (std::is_same_v<T, double>)
            m_os << std::fixed << std::setprecision(m_dprec);

        m_os << arg; // m_os can already work with arg
    }, v);

    (m_os << std::setprecision(def_prec)).flags(def_flags); // reset
    m_os << m_delim;
    return *this;
}
```

06-basic-data-types-resurrections/basic-data-types-resurrections-03.cpp

Everything happens inside the `operator =` method and the reason is that extracting the type from an `std::variant` is not that difficult, because we have the help of new and known friends. First

- `std::visit` that will put into a function of our choosing the argument inside the `std::variant` (or a collection of those)

If we use a generic *lambda*, i.e.: with an `auto` parameter, the type will be worked out for us. But we still need the type and:

- `decltype` comes to the rescue to get the type of `auto &arg`
- Coupled with a new friend: `std::decay_t`. This fine piece of machinery removes the *reference* and gives us the real type behind `auto &arg`

With the type in the hand, we can again resort to using our compile-time friend and wonder, `if constexpr`, to apply specific code for `float` and `double` as we did in the past.

There is something worth mentioning, the `template <typename V>` technique. Because that `V` is the parameter to the method. Recall that the input methods will return an `std::variant` with the value embedded. Our `operator =` does not need to know the types in the template instantiation; this will be worked out by the compiler.

What we could do is add a bit of *SFINAE* to make sure that `V` is an `std::variant`, using `std::enable_if_t` and a check for the type.

Let us see the complete code of this final solution.

```
template<typename V>
auto &operator =(const V &v) const {
    const auto def_prec{m_os.precision()}; // save current prec
    const auto def_flags{m_os.flags()}; // save current flags

    // visit our variant val and get the type in auto format
    std::visit([&](auto &arg) {  // arg is the value inside variant v
        using T = std::decay_t<decltype(arg)>; // deduce actual type
        // do something if float or double
        if constexpr (std::is_same_v<T, float>)
            m_os << std::fixed << std::setprecision(m_fprec);
        else if constexpr (std::is_same_v<T, double>)
            m_os << std::fixed << std::setprecision(m_dprec);

        m_os << arg; // m_os can already work with arg
    }, v);

    (m_os << std::setprecision(def_prec)).flags(def_flags); // reset
```

```
        m_os << m_delim;
        return *this;
    }
```

06-basic-data-types-resurrections/basic-data-types-resurrections-03.cpp

Done and dust. We have really generic code for both `vistream_iterator` and `vostream_iterator`, both freed from having to have knowledge of the `std::variant`.

6.6. Summary

The goal in this chapter was to solve the conundrum of using iterators to iterate over types, rather than fully iterate over the types at compile time. And our solutions have also been subject to iteration until we reached that goal, as much as possible.

6.7. Bonus Points

OK, if you have made it that far, you have seen the cliffhangers about adding *SFINAE* for the new things. We could not really leave the chapter without adding it.

First, we need to check if the code generating the *lambda* for type conversion from an `std::istream` is really getting an `std::variant`. This is the trickiest because one cannot really check for `std::variant` because it will always be bound to a template pack.

The trick is therefore to specialize a `std::false` / `std::true` check, where the true part is a specialization containing an `std::variant<Args...>`.

```
// SFINAE to check for a variant
template<typename V>
struct is_variant_t : std::false_type {};

// Need to specialize to consider a template pack, for std::variant
template<typename ...Args>
struct is_variant_t<std::variant<Args...>> : std::true_type {};

template<typename V>
constexpr bool is_variant_v = is_variant_t<V>::value;

template<typename V>
using enable_if_variant = std::enable_if_t<is_variant_v<V>>*;
```

```
template <typename Variant, size_t i = 0, enable_if_variant<Variant> = nullptr>
auto
for_each_in(InTypeProcs &itp) {
```

06-basic-data-types-resurrections/basic-data-types-resurrections-04.cpp

`vistream_iterator` is the next one in need of *SFINAE* to see if the template parameter `I` is really an *InputIterator* and even more: if it delivers a function returning an `std::variant` after being dereferenced and called with an `std::istream`. Because that is the expectation when `vistream_iterator` is dereferenced itself.

```
// SFINAE to check for I being an Input iterator and delivering a variant
template <typename T, typename Tag>
constexpr bool is_it_tag_v = std::is_base_of_v<
    Tag, typename std::iterator_traits<T>::iterator_category>;

template <typename I>
constexpr bool is_input_v = is_it_tag_v<I, std::input_iterator_tag>;

template <typename I>
constexpr bool input_variant_v = is_variant_v<
    typename std::decay_t<decltype(*std::declval<I>())>::result_type>;

template <typename I>
constexpr bool is_input_variant_v = is_input_v<I> && input_variant_v<I>;

template<typename I>
using enable_if_input_variant = std::enable_if_t<is_input_variant_v<I>>*;

template <typename I, enable_if_input_variant<I> = nullptr>
struct vistream_iterator {
```

06-basic-data-types-resurrections/basic-data-types-resurrections-04.cpp

Recall that lately we had made a new friend, `std::decay_t<T>`, and it comes to the rescue again. But not alone, because it needs a push from the left, i.e.: being prefixed by `typename` to be able to finally get down to `::result_type`.

This is because `::result_type` is a dependent name itself, only available after the previous conundrum has been solved. `std::function` stores there the return type of the stored function, and that is what we are looking for.

After that comes our `template <V> vostream::operator =(const V &v) const`. That `V` needs to be a variant. We therefore reuse what we already developed above and apply `enable_if_variant<V>`.

```
template<typename V, enable_if_variant<V> = nullptr>
auto &operator =(const V &v) const {
```

06-basic-data-types-resurrections/basic-data-types-resurrections-04.cpp

For the general case, we have reapplied the rest of the standard *SFINAE* machinery, to check for input/output iterators that also have compatible types and was being used in the first solution.

It is now that we have a really complete solution. Game over and here is the code.

```
#include <any> // std::any
#include <algorithm> // std::copy_n
#include <functional> // std::function
#include <iomanip> // std::setprecision
#include <ios> // std::fixed
#include <iostream> // std::cin/cout
#include <iterator> // std::ostream_iterator/istream
#include <type_traits> // std::is_same_v
#include <variant> // std::variant
#include <vector> // std::vector

using BasicTypes = std::variant<int, long, char, float, double>;
using InTypeProc = std::function<BasicTypes(std::istream &)>;
using InTypeProcs = std::vector<InTypeProc>;

static InTypeProcs intypeprocs;

// SFINAE to check for a variant
template<typename V>
struct is_variant_t : std::false_type {};

// Need to specialize to consider a template pack, for std::variant
template<typename ...Args>
struct is_variant_t<std::variant<Args...>> : std::true_type {};

template<typename V>
```

```
constexpr bool is_variant_v = is_variant_t<V>::value;

template<typename V>
using enable_if_variant = std::enable_if_t<is_variant_v<V>>*;

template <typename Variant, size_t i = 0, enable_if_variant<Variant> = nullptr>
auto
for_each_in(InTypeProcs &itp) {
    using BasicType = std::variant_alternative_t<i, Variant>; // get type
    auto intypeproc = [](std::istream &is) {
        return Variant{*std::istream_iterator<BasicType>{is}};
    };
    itp.push_back(intypeproc);

    if constexpr ((i + 1) < std::variant_size_v<Variant>)
        for_each_in<Variant, i + 1>(itp);
}

// SFINAE to check for I being an Input iterator and delivering a variant
template <typename T, typename Tag>
constexpr bool is_it_tag_v = std::is_base_of_v<
    Tag, typename std::iterator_traits<T>::iterator_category>;

template <typename I>
constexpr bool is_input_v = is_it_tag_v<I, std::input_iterator_tag>;

template <typename I>
constexpr bool input_variant_v = is_variant_v<
    typename std::decay_t<decltype(*std::declval<I>())>::result_type>;

template <typename I>
constexpr bool is_input_variant_v = is_input_v<I> && input_variant_v<I>;

template<typename I>
using enable_if_input_variant = std::enable_if_t<is_input_variant_v<I>>*;

template <typename I, enable_if_input_variant<I> = nullptr>
struct vistream_iterator {
    // Iterator tags
    using iterator_category = std::input_iterator_tag;
    using difference_type   = std::ptrdiff_t;
    using value_type        = std::any;
    using reference         = value_type &;
    using pointer           = value_type *;
```

```
    std::istream &m_is = std::cin;
    I m_first, m_last;
    bool m_end = true;

    vistream_iterator(std::istream &is, I first, I last)
        : m_is{is}, m_first{first}, m_last{last}, m_end{first == last} {}
    vistream_iterator(I last) : m_first{last}, m_last{last} {}

    auto operator ->() const noexcept { return this; }
    auto operator *() const {
        return (*m_first)(m_is); // fetched intypeproc delivers result
    }
    // Prefix increment
    auto &operator ++() {
        m_end = ((++m_first) == m_last);
        return *this;
    }
    // Postfix increment
    auto operator ++(int) const {
        auto tmp = *this;
        ++(*this);
        return tmp;
    }

    auto operator ==(const vistream_iterator& o) const {
        return m_end ? o.m_end : (o.m_end ? false : m_first == o.m_first);
    };
    auto operator !=(const vistream_iterator& other) const {
        return not (*this == other);
    }
};

////////////////////////////////////////////////////////////////////////////
class vostream_iterator {
public:
    // needed for an iterator
    using iterator_category = std::output_iterator_tag;
    using difference_type   = std::ptrdiff_t;
    using value_type        = std::any;
    using reference         = value_type &;
    using pointer           = value_type *;

private:
```

```
    std::ostream &m_os;
    std::string m_delim;
    size_t m_fprec;
    size_t m_dprec;

public:
    // constructor replicating the wrapped iterator's constructor
    vostream_iterator(std::ostream &os, const std::string &delim,
        size_t fprec = 3, size_t dprec = 9)
        : m_os{os}, m_delim{delim}, m_fprec{fprec}, m_dprec{dprec} {}

    // no-ops because only the assignment (= operator does something)
    auto operator ->() const { return this; };
    auto &operator *() const { return *this; }
    auto &operator ++() { return *this; } // ++prefix
    auto &operator ++(int) { return *this; } // postfix++

    template<typename V, enable_if_variant<V> = nullptr>
    auto &operator =(const V &v) const {
        const auto def_prec{m_os.precision()}; // save current prec
        const auto def_flags{m_os.flags()}; // save current flags

        // visit our variant val and get the type in auto format
        std::visit([&](auto &arg) {  // arg is the value inside variant v
            using T = std::decay_t<decltype(arg)>; // deduce actual type
            // do something if float or double
            if constexpr (std::is_same_v<T, float>)
                m_os << std::fixed << std::setprecision(m_fprec);
            else if constexpr (std::is_same_v<T, double>)
                m_os << std::fixed << std::setprecision(m_dprec);

            m_os << arg; // m_os can already work with arg
        }, v);

        (m_os << std::setprecision(def_prec)).flags(def_flags); // reset
        m_os << m_delim;
        return *this;
    }
};

// SFINAE (extra, input is taken from above) for the general solution
template <typename O>
constexpr bool is_output_v = is_it_tag_v<O, std::output_iterator_tag>;
```

```
template<typename I, typename O>
constexpr bool io_iterators_v = is_input_v<I> && is_output_v<O>;

template <typename I, typename O>
using io_type = decltype(*std::declval<O>() = *std::declval<I>());

template<typename, typename, typename = void>
struct io_i2o : std::false_type {};

template<typename I, typename O>
struct io_i2o<I, O, std::void_t<io_type<I, O>>>
    : std::true_type {};

template<typename I, typename O>
constexpr bool io_i2o_v = io_i2o<I, O>::value;

template<typename I, typename O>
using enable_if_io = std::enable_if_t<io_iterators_v<I, O> && io_i2o_v<I, O>>*;

// Solution
template<typename I, typename O, enable_if_io<I, O> = nullptr>
auto
basic_types(I first, I last, O out) {
    return std::copy(first, last, out);
}

int
main(int, char *[]) {
    for_each_in<BasicTypes>(intypeprocs);
    basic_types(
        vistream_iterator{std::cin, intypeprocs.begin(), intypeprocs.end()},
        vistream_iterator{intypeprocs.end()},
        vostream_iterator{std::cout, "\n"}
    );
    return 0;
}
```

06-basic-data-types-resurrections/basic-data-types-resurrections-04.cpp

7. For Loop

Title: For Loop **Link**: https://www.hackerrank.com/challenges/c-tutorial-for-loop/problem

Let us go for the next "inconsequential" challenge that we want to transform in the best next iterating challenge.

This one is a `for` loop in between the integers `a` and `b`, where those integers are read from `std::cin`. The output yields the equivalent integer as a word for anything `< 10` or else the parity of the given integer. An obvious approach to the problem would be as follows.

```
#include <iostream> // std::cout

int
main(int, char *[]) {
    const char *numbers[] = { // we do not need the zero index, a >= 1
        "even", "one", "two", "three", "four",
        "five", "six", "seven", "eight", "nine", "odd"
    };

    int a, b;
    std::cin >> a >> b;
    for(int i = a; i <= b; i++)
        std::cout << numbers[i < 10 ? i : 10 * (i % 2)] << std::endl;

    return 0;
}
```

07-for-loop/for-loop-01.cpp

To be really smart we use the left and right sides of the `const char *numbers[]` array, to have our `even` and `odd` results all stored within a single data structure.

7.1. Going Python

Using some `std::istream_iterator<int>` and `std::ostream_iterator<std::string>` is not what we are looking for. Our goal has to be an iteration that removes the `for` loop and let us iterate from `a` to `b`, i.e.: we need something that generates all the integers.

We obviously do not want to create an `std::vector` that holds all the integers even if in the test case that goes with the code we have `8` and `11`. That will, for sure, not mean any kind of

stress, but we could be considering millions of integers and the question is why should be storing them. If we consider Python, the builtin `range` can be used. For example (simplified example)

```
`\n'.join(numbers[i if i < 10 else 10 * (i % 2)] for i in range(a, b + 1))
```

Ranges in Python are half-open, hence the `b + 1` and the integer are not generated, then stored and finally delivered one by one. The function generates and yields an integer, delivering the next when requested. We know (living in the future) that C++20 has implemented something like this with: `std::views::iota(1, 10)` but recall we are still on C++17.

Let us therefore address the problem of "replicating" (what we need) the `range` function in C++. Because we want to pass the range to a function that transform the integers to a string, we need a range, i.e.: `first` and `last` iterators, that we will get from `begin` and `end` functions. What we are basically describing is a container, and in our case we are describing a *"virtual"* container because we are not going to contain anything, except the description of the range: `start`, `stop`. Although we could hardcode the step-by-step increment between those limits, let us be flexible by defining also a `step`, that will default to a value of `1`.

This is how the beginning of our container looks like.

```
template <typename T=int>
class Range {
    struct StartStopStep {
        const T start = 0;
        const T stop = std::numeric_limits<T>::max();
        const T step = 1;
    } m_sss;
```

07-for-loop/for-loop-02.cpp

Notice how, right from the start, our container is a template defaulting to `int`. The idea is to allow using different types of integral types. To disallow using something like a `float`, we will later consider *SFINAE*. To make the range as *unlimited* as possible, the default value for the `stop` part is taken from the standard library with `std::numeric_limits<T>::may()`. Much better than writing fixed values.

The constructors are the next best addition to the sauce of this container.

1. We have a `struct StartStopStep` defined, so let us use it as a direct input for a constructor. That would be akin to using a `slice` in *Python*.
2. Given that we have already referred to *Python* a couple of times, nothing like defining constructors, that mimic the way the builtin `range` works.

```
- Number of parameters: `1`. The value is used for `stop`. Our other parameters
take default values, i.e.: `start = 0` and `step = 1`

- Number of parameters: `2`. These are then, in order, `start` and `stop`, with
`step` again defaulting to be `1`

- Number of parameters: `3`. These are then, in order, `start`, `stop` and
`step`be `1`
```

Although we have three options for constructor family taking integers, we only need to define two of them. Let us see it.

```
public:
    Range(const StartStopStep& sss) : m_sss{sss} {};
    Range(const T stop) : m_sss{.stop=stop} {};
    Range(const T start, const T stop, const T step = 1)
        : m_sss{.start=start, .stop=stop, .step=step} {};
```

07-for-loop/for-loop-02.cpp

The last constructor covers the `2-parameters` and `3-parameters`, assigning a default value to `step`. This last constructor shows us something very interesting: a pseudo-parameter-by-name calling convention.

```
    Range(const T stop) : m_sss{.stop=stop} {};
    Range(const T start, const T stop, const T step = 1)
        : m_sss{.start=start, .stop=stop, .step=step} {};
```

07-for-loop/for-loop-02.cpp

Our internal member `m_sss`, an instance of `StartStopStep` is being initialized by referring to

the members defined in it, just by prefixing the name with a `.` (a dot). It feels almost like *Python* but it is not. The magic is limited, but it is a nice example of how *"named parameters"* can make things very clear. Consider that *Python* ended up adding more types of parameters, like positional-only parameters, named-only parameters to give users a finer grained control when calling a function.

It is now time to see how this container defines the iterator that will go over the `StartStopRange`.

```
private:
    struct Iter {
    private:
        StartStopStep m_sss;
        T m_pos;

    public:
        // Iterator tags
        using iterator_category = std::forward_iterator_tag;
        using difference_type   = std::ptrdiff_t;
        using value_type        = T;
        using reference         = value_type; // usually value_type &
        using pointer           = value_type; // usually value_type *

        Iter(const StartStopStep &sss) : m_sss{sss}, m_pos{m_sss.start} {};
        Iter(const T &pos) : m_sss{.stop=pos}, m_pos{pos}  {}; // en-of-range

        auto operator *() const { return m_pos; }
        auto operator ->() const { return m_pos; }
        auto& operator ++() { // Prefix increment - increase until stop
            m_pos = std::min(m_pos + m_sss.step, m_sss.stop);
            return *this;
        }
        // Postfix increment
        auto operator ++(int) { Iter tmp = *this; ++(*this); return tmp; }

        auto operator ==(const Iter& o) const { return m_pos == o.m_pos; }
        auto operator !=(const Iter& o) const { return m_pos != o.m_pos; }
    };
```

07-for-loop/for-loop-02.cpp

The iterator needs to know how the range is defined and it will therefore hold a copy of it.

We could have achieved the same effect by keeping a pointer to the container that instantiated the iterator. For practical purposes it is the same. But notice that if we do not hold that pointer, the container could be deleted and the iterator can still be valid.

Something that seems contradictory because if there is no container, after the deletion, one could ask what is what the iterator is iterating on. In our case, the answer is easy: on the virtual range `StartStopStep`, that is **not contained** anywhere, but rather **defined** in the container. If the iterator makes a copy of the range definition, it can directly use that definition to iterate.

The rest should be straightforward. There is an `m_pos` member that will control where the iterator is, used at the same time to directly hold the final position when instantiating the end of range iterator through the method `end`. The iterator will increase `m_pos`, if still possible, on each iteration, until the value defined by `stop` is reached.

It is time to show the entire solution. But before that, let us explain the other things we did. We have also:

- Gathered input with an `std::istream_iterator<int>(std::cin)`, creating `auto` variables for `a` and `b`.
- Used our container `Range` to define the iteration range `[a, b]`, which is a closed range including both `a` and `b`.

  Recall that we are emulating the *Python* version and our definition is half-open, `[a, b)`. Hence the need to instantiate the container with `range = Range(a, b + 1)`.

- Put our solution in a separate method that takes an input range with `first` and `last` iterators and a destination `out` iterator.
- Invoked our solution with `range.begin()` and `range.end()` and our already classic output iterator, i.e.: `std::ostream_iterator<std::string>(std::cout, "\n")`.

In our solution method we have done two things that differ from the previous simplistic approach. Because our objective is to reuse what people have developed over the years for the *STL*, we:

- Changed the storage for the words to be an `std::array`. Recall that with C++17 we can abuse the compiler and avoid telling how many elements there are and what the type of the elements is.
- Put the logic to decide the output word in a small *lambda expression*.

- We need it to be able to use `std::transform(first, last, destination, unary_predicate)`. I.e.: our range will be iterated and each value will be passed as the single argument (*"unary"*) to our *lambda*/function (*"predicate"*). The result of the evaluation will be sent to the destination.

Our case is simple and things go to the standard output but we could be sending the result to an `std::vector` or the iterator could point to a network socket or send an *HTTP* request to a well-known *API*.

Here is the code.

```
#include <algorithm> // std::transform
#include <array> // std::array
#include <iostream> // std::cin/cout
#include <iterator> // std::istream/ostream_iterator
#include <limits> // std::numeric_limits

template <typename T=int>
class Range {
    struct StartStopStep {
        const T start = 0;
        const T stop = std::numeric_limits<T>::max();
        const T step = 1;
    } m_sss;

public:
    Range(const StartStopStep& sss) : m_sss{sss} {};
    Range(const T stop) : m_sss{.stop=stop} {};
    Range(const T start, const T stop, const T step = 1)
        : m_sss{.start=start, .stop=stop, .step=step} {};

private:
    struct Iter {
    private:
        StartStopStep m_sss;
        T m_pos;

    public:
        // Iterator tags
        using iterator_category = std::forward_iterator_tag;
        using difference_type   = std::ptrdiff_t;
        using value_type        = T;
```

```
        using reference         = value_type; // usually value_type &
        using pointer           = value_type; // usually value_type *

        Iter(const StartStopStep &sss) : m_sss{sss}, m_pos{m_sss.start} {};
        Iter(const T &pos) : m_sss{.stop=pos}, m_pos{pos}  {}; // en-of-range

        auto operator *() const { return m_pos; }
        auto operator ->() const { return m_pos; }
        auto& operator ++() { // Prefix increment - increase until stop
            m_pos = std::min(m_pos + m_sss.step, m_sss.stop);
            return *this;
        }
        // Postfix increment
        auto operator ++(int) { Iter tmp = *this; ++(*this); return tmp; }

        auto operator ==(const Iter& o) const { return m_pos == o.m_pos; }
        auto operator !=(const Iter& o) const { return m_pos != o.m_pos; }
    };

public:
    auto begin() const { return Iter{m_sss}; } // copy range, pos at start
    auto end() const { return Iter{m_sss.stop}; } // place directly at end
};

constexpr auto numbers = std::array{ // we do not need the zero index, a >= 1
    "even", // index 0
    "one", "two", "three", "four", "five", "six", "seven", "eight", "nine",
    "odd" // index 10
};

template <typename I, typename O>
auto
num2words(I first, I last, O out) {
    auto n2w = [](auto i) { return numbers[i < 10 ? i : 10 * (i % 2)]; };
    std::transform(first, last, out, n2w);
}

int
main(int, char *[]) {
    auto in = std::istream_iterator<int>{std::cin}; // prepare input iterator
    auto a = *in++, b = *in++; // gather parameters
    auto range = Range{a, b + 1};  // our range is half-open, need extra +1
    num2words(
        range.begin(), range.end(),
```

```
        std::ostream_iterator<std::string>{std::cout, "\n"}
    );

    return 0;
}
```

07-for-loop/for-loop-02.cpp

7.2. Refining And SFINAE

Let us start with our `Range<T = int>`. Ideally we would accept anything that works like an integral type. For obvious reasons a `float` (or a `double`) may lead to failures due to the non-exact nature of floating point arithmetic. With that in mind, adding *SFINAE* for `Range<T = int>` looks like this.

```
template <typename T>
using enable_if_integral = std::enable_if_t<std::is_integral_v<T>>;

template <typename T = int, typename = enable_if_integral<T>>
class Range {
```

07-for-loop/for-loop-03.cpp

Note that we have change how we integrate our custom `enable_if`. Instead of delivering a type that will have a default value, `enable_if<x, y> = nullptr`, we use the return type as the input to an anonymous template parameter, i.e.:

```
template <typename T = int, typename = enable_if<x, y>>
```

Just a different way to use our *SFINAE* machinery.

There is something that we have to take into account because `std::is_integral<T>` will recognize integral types. However, user defined classes that work like integral types will not pass the test.

We would need this other check.

```
template <typename T>
```

```
using enable_if_integral = std::enable_if_t<std::numeric_limits<T>::is_integer;

template <typename T = int, typename = enable_if_integral<T>>
class Range {
```

With a specialization added to recognize our user defined type. `std::is_integral` has an *undefined behavior* if specializations are added.

Furthermore, we also have this at the beginning of `Range`.

```
    struct StartStopStep {
        const T start = 0;
        const T stop = std::numeric_limits<T>::max();
        const T step = 1;
    } m_sss;
```

07-for-loop/for-loop-03.cpp

I.e.: we source the maximum value of our integral type `std::numeric_limits<T>::max()` and we would need a similar construct (or a specialization) for our user defined `T`. Too much for the scope of our work.

With regards to the iterators, we could simply add the previous *SFINAE* machinery, fire and forget. However, we have something we can improve: our solution function could take iterators and a custom function to solve the problem. And that means we also have to check the signature of that function: parameters and return type. This is how we do it.

```
template <typename T>
using it_type = typename std::iterator_traits<T>::value_type;

template <typename I, typename F>
using i2f_type = std::invoke_result_t<F, it_type<I>>;

template<typename, typename, typename, typename = void>
constexpr bool i2f2o_v = false;

template<typename I, typename O, typename F>
constexpr bool i2f2o_v<I, O, F,
    std::void_t<decltype(std::declval<O>() = std::declval<i2f_type<I, F>>())>
    >
```

```
    = true;
```

07-for-loop/for-loop-03.cpp

We first fetch the type of the input iterator with `std::iterator_traits<T>::value_type` as an alternative to our previous formulation with `decltype(*std::declval<T>())`. This is what goes as the parameter to the function provided for the solution.

To get the return type we meet a new helping hand: `std::invoke_result<F, Args ...>` a new entry in C++17. Given a callable `F` and a set of types that will be provided as arguments for the call, we find out the return type at compile time.

With those two things in hand and having also an output iterator `O`, we use the `constexpr bool` templated setup to find out if the output iterator can take what the function returns after taking the type. For that we use again `std::void_t<type, ...>`. Recall that it will turn valid types into `void` and fail otherwise. Our type will be valid if our `std::declval` formulation inside `decltype` is valid: `std::declval<O>() = std::declval<i2f_type<I, F>>()`. I.e.: if the `operator =` of our output iterator can take the return type of the solution function, given the proper input type.

We put it all together with the standard checks to see if our template parameters are iterators (or at least look like them) and coupled with `std::enable_if`, we have a good barrier for wrong inputs.

```
template <typename I, typename O, typename F>
using enable_if_iof = std::enable_if_t<io_iterators_v<I, O> && i2f2o_v<I, O, F>>;

template <typename I, typename O, typename F, typename = enable_if_iof<I, O, F>>
auto
num2words(I first, I last, O out, F n2w) {
    std::transform(first, last, out, n2w);
}
```

07-for-loop/for-loop-03.cpp

The full code contains four alternative formulations of functions for the solutions. Only one solves the challenge but that is not he point. The point being: two of them meet the *SFINAE* requirements and two of them fail them (wrong number of parameters and wrong return type) Feel free to play with them. The default is the function that solves the problem.

Here is the full code.

```
#include <algorithm> // std::transform
#include <array> // std::array
#include <iostream> // std::cin/cout
#include <iterator> // std::istream/ostream_iterator
#include <limits> // std::numeric_limits
#include <type_traits> // std::enable_if, std::is_integral, std::void_t

template <typename T>
using enable_if_integral = std::enable_if_t<std::is_integral_v<T>>;

template <typename T = int, typename = enable_if_integral<T>>
class Range {
    struct StartStopStep {
        const T start = 0;
        const T stop = std::numeric_limits<T>::max();
        const T step = 1;
    } m_sss;

public:
    Range(const StartStopStep& sss) : m_sss{sss} {};
    Range(const T stop) : m_sss{.stop=stop} {};
    Range(const T start, const T stop, const T step = 1)
        : m_sss{.start=start, .stop=stop, .step=step} {};

private:
    struct Iter {
    private:
        StartStopStep m_sss;
        T m_pos;

    public:
        // Iterator tags
        using iterator_category = std::forward_iterator_tag;
        using difference_type   = std::ptrdiff_t;
        using value_type        = T;
        using reference         = value_type; // usually value_type &
        using pointer           = value_type; // usually value_type *

        Iter(const StartStopStep &sss) : m_sss{sss}, m_pos{m_sss.start} {};
        Iter(const T &pos) : m_sss{.stop=pos}, m_pos{pos}  {}; // end-of-range
```

```
        auto operator *() const { return m_pos; }
        auto operator ->() const { return &m_pos; }
        auto& operator ++() { // Prefix increment - increase until stop
            m_pos = std::min(m_pos + m_sss.step, m_sss.stop);
            return *this;
        }
        // Postfix increment
        auto operator ++(int) { Iter tmp = *this; ++(*this); return tmp; }

        auto operator ==(const Iter& o) const { return m_pos == o.m_pos; }
        auto operator !=(const Iter& o) const { return m_pos != o.m_pos; }
    };

public:
    auto begin() const { return Iter{m_sss}; } // copy range, pos at start
    auto end() const { return Iter{m_sss.stop}; } // place directly at end
};

// SFINAE to check for I being an Input iterator and delivering a variant
template <typename T, typename Tag>
constexpr bool is_it_tag_v =
    std::is_base_of_v<Tag, typename std::iterator_traits<T>::iterator_category>;

template <typename I>
constexpr bool is_input_v = is_it_tag_v<I, std::input_iterator_tag>;

template <typename O>
constexpr bool is_output_v = is_it_tag_v<O, std::output_iterator_tag>;

template<typename I, typename O>
constexpr bool io_iterators_v = is_input_v<I> && is_output_v<O>;

template <typename T>
using it_type = typename std::iterator_traits<T>::value_type;

template <typename I, typename F>
using i2f_type = std::invoke_result_t<F, it_type<I>>;

template<typename, typename, typename, typename = void>
constexpr bool i2f2o_v = false;

template<typename I, typename O, typename F>
constexpr bool i2f2o_v<I, O, F,
    std::void_t<decltype(std::declval<O>() = std::declval<i2f_type<I, F>>())>
```

```
    >
    = true;

template <typename I, typename O, typename F>
using enable_if_iof = std::enable_if_t<io_iterators_v<I, O> && i2f2o_v<I, O, F>>;

template <typename I, typename O, typename F, typename = enable_if_iof<I, O, F>>
auto
num2words(I first, I last, O out, F n2w) {
    std::transform(first, last, out, n2w);
}

// SFINAE is over
constexpr auto numbers = std::array{ // we do not need the zero index, a >= 1
    "even", // index 0
    "one", "two", "three", "four", "five", "six", "seven", "eight", "nine",
    "odd" // index 10
};

int
main(int, char *[]) {
    auto in = std::istream_iterator<int>{std::cin}; // prepare input iterator
    auto a = *in++, b = *in++; // gather parameters
    auto range = Range{a, b + 1};  // our range is half-open, need extra +1

#if 0
    // SFINAE => NOK - 1 parameters but returns int
    auto n2w = [](auto i) { return i; };
#elif 0
    // SFINAE => OK - 1 parameter and returns string
    auto n2w = [](auto i) { return std::string{}; };
#elif 0
    // SFINAE => NOK - 2 parameters instead of 1
    auto n2w = [](auto i, auto x) { return std::string{}; };
#else // Default - The problem solution
    // SFINAE => OK - 1 param, returns convertible to std::string
    auto n2w = [](auto i) { return numbers[i < 10 ? i : 10 * (i % 2)]; };
#endif

    num2words(
        range.begin(), range.end(),
        std::ostream_iterator<std::string>{std::cout, "\n"},
        n2w
    );
```

```
    return 0;
}
```

07-for-loop/for-loop-03.cpp

Colophon

© Year Daniel Rodriguez

All rights reserved.

Except where allowed by copyright laws, neither the book in full nor portions of it may be reproduced, whatever the form, if no written permission from the publisher/author has been granted.

As much as possible and within the limits of reality, this book tries to provide accurate information on the subject. During the preparation, the author and publisher have striven to achieve and error-free content, but some errors may still be lurking in between the lines. Moreover, speculative theories are just speculations based on the facts surrounding them, with no implied certainty of truth.

1st Edition - June 2025

Daniel Rodriguez is a hobby programmer trying to keep up to date with the latest C++ standards.

This book is just a collection of simple problems that have been solved with an incredible amount of complexity by iteratively adding Iterators, SFINAE and whatever the author could come up with.

Even a simple “Hello, World!” is the target of this futile attempt to show some actual C++ knowledge.

This self-inflicted punishment is geared towards leaving a permanent mark in the brain in the form of C++ concepts.

If you read, or peruse, this book and find something interesting in it, the author will be more than pleased to have contributed not only to his own C++ improvement but to that of someone else.

If on the other hand, you find incredible mistakes and flawed reasoning, do not hesitate to go to the Github repository and point it out. Your humble author will be more than delighted to learn something new.